

Documentazione progetto

Paolo Capelli, Alessio Giovanelli e Riccardo Maffeis

Matricole n. 1081221, 1081610 e 1085706



Università degli Studi di Bergamo

Laurea Magistrale in Ingegneria Informatica

Corso di Progettazione, Algoritmi e Computabilità (9 CFU)

Sommario

Iterazione 0	6
Introduzione e panoramica del sistema	6
Requisiti funzionali e analisi dei casi d'uso.....	7
Topologia del sistema.....	8
Design Pattern	8
Toolchain e tecnologie.....	9
Iterazione 1	10
Introduzione	10
UC1.1: Registrazione.....	10
UC1.2: Login.....	10
UC1.3: Selezione preferenze	11
UC1.4: Logout.....	11
UC2.1: Accesso alla piattaforma	11
UC3.1: Salvataggio dati	12
Diagramma delle componenti UML	12
Diagramma delle classi per le interfacce	13
Diagramma delle classi per tipi di dato	14
Diagramma di deployment.....	14
Testing	15
Analisi statica.....	15
Analisi dinamica.....	15
Documentazione API	16
Iterazione 2	19
Introduzione	19
UC4.4: Ricevere consegna prenotazione	20
UC4.5: Consultazione dello storico prenotazioni.....	20
UC5.1: Monitoraggio stato parcheggio	21
UC5.2: Gestione della manutenzione parcheggio	21
Diagramma delle componenti UML	22
Diagramma delle classi per le interfacce	22
Diagramma delle classi per tipi di dato	23

Diagramma di deployment.....	23
Testing	24
Analisi statica.....	24
Analisi dinamica.....	24
Documentazione API	26
Iterazione 3	30
Introduzione	30
UC 6.1: Seguire istruzione per arrivare al parcheggio	30
UC 6.2: Gestione prenotazione	30
UC 7.1: Visualizzazione simulazione operativa	31
UC 7.2: Validazione QR Code e Accesso	31
UC 7.3: Consultazione Cronologia Giornaliera	32
UC 8.1: Modifica Manuale Disponibilità	32
UC 8.2: Blocco Emergenza Parcheggio	32
UC9: Algoritmo di ottimizzazione selezione parcheggio.....	33
UC10: Algoritmo di calcolo tariffa prenotazione	38
UC11: Algoritmo di identificazione percorso migliore.....	42
Diagramma delle componenti UML	48
Diagramma delle classi per le interfacce	48
Diagramma delle classi per tipi di dato	49
Diagramma di deployment.....	49
Testing	50
Analisi statica.....	50
Analisi dinamica.....	50
Documentazione API	52

Elenco delle equazioni

Calcolo della complessità algoritmo assegnazione posto ottimo	34
Calcolo della complessità algoritmo calcolo importo permanenza	39
Calcolo della complessità algoritmo di identificazione percorso migliore	43

Elenco dei diagrammi

Flowchart dell'algoritmo assegnazione posto ottimo.....	37
Flowchart dell'algoritmo calcolo importo permanenza.....	41
Flowchart dell'algoritmo identificazione percorso migliore	47

Elenco delle figure

Diagramma UML dei casi d'uso.....	7
Topologia del sistema	8
Diagramma delle componenti UML.....	13
Diagramma delle classi UML per interfacce	13
Diagramma delle classi UML per tipi di dato	14
Diagramma di deployment UML.....	14
API per la registrazione dell'utente	16
API per il login dell'utente.....	17
API per aggiornare le preferenze	17
API per il login dell'operatore	18
Diagramma delle componenti UML.....	22
Diagramma delle classi UML per interfacce.....	22
Diagramma delle classi UML per tipi di dato	23
Diagramma di deployment UML.....	23
Esito test JUnit sulla classe AnaliticheController.....	24
Esito test JUnit sulla classe AnaliticheServiceImpl.....	24
Esito test JUnit sulla classe LogController	25
Esito test JUnit sulla classe LogServiceImpl.....	25
Esito test JUnit sulla classe ParcheggioController	25
Esito test JUnit sulla classe ParcheggioServiceImpl	26
Esito test JUnit sulla classe PrenotazioneController	26
Esito test JUnit sulla classe PrenotazioneServiceImpl	26
Coverage fornita dai test JUnit del progetto.....	26
API per la ricerca dei parcheggi in base all'area.....	27
API per la prenotazione di un posto nel parcheggio	27
API per recuperare lo storico delle prenotazioni dell'utente	28
API per recuperare le analitiche del parcheggio	28
API per recuperare i log di un'analitica.....	29
Diagramma delle componenti UML.....	48

Diagramma delle classi UML per interfacce	48
Diagramma delle classi UML per tipi di dato	49
Diagramma di deployment UML.....	49
Esito test JUnit sulla classe PostoController	50
Esito test JUnit sulla classe PostoServiceImpl	50
Esito test JUnit sulla classe PrenotazioneController	51
Esito test JUnit sulla classe PrenotazioneServiceImpl	51
Coverage fornita dai test JUnit del progetto.....	51
Esito integration test JUnit sulle classi degli algoritmi	52
Esito integration test Flutter sulla classe dell'algoritmo	52
API per la ricerca dei posti nel parcheggio	53
API per aggiornare la disponibilità di un posto in un parcheggio	53
API per l'emissione dell'importo da pagare	54
API per la ricerca di prenotazioni tramite QRcode	54
API per la ricerca delle prenotazioni nel parcheggio	55

Elenco delle tabelle

Casi d'uso iterazione 1	7
Casi d'uso iterazione 2	7
Casi d'uso iterazione 3.....	8
Toolchain e tecnologie	9

Iterazione 0

Introduzione e panoramica del sistema

Il progetto vede la realizzazione di un sistema informatico per la gestione di un parcheggio multipiano a pagamento, con l'obiettivo di ottimizzare l'uso degli spazi, automatizzare i processi di accesso/uscita e semplificare la gestione economica del servizio.

Il sistema deve integrare una componente Desktop destinata agli operatori di cassa e una componente Web dedicata agli utenti, che potranno verificare la disponibilità dei posti e prenotare in anticipo.

Il sistema principale sarà utilizzato dal personale del parcheggio per:

- Controllare le barre di accesso e uscita;
- Monitorare e aggiornare in tempo reale la disponibilità dei posti;
- Gestire i blocchi fisici (manutenzione o posti riservati);
- Emettere e annullare i ticket di parcheggio;
- Calcolare automaticamente la tariffa in base al tempo di permanenza del veicolo;
- Gestire la cassa e i pagamenti, registrando ogni transazione.

Il software dovrà quindi fornire un'interfaccia intuitiva e sicura per l'operatore, consentendo la supervisione e l'intervento sui dispositivi automatici del parcheggio.

La componente Web del sistema è destinata agli utenti che desiderano:

- Consultare la disponibilità dei posti in tempo reale;
- Effettuare la prenotazione di un posto in una data e fascia oraria prefissata;
- Ricevere la conferma della prenotazione (voucher elettronico o QR code da presentare all'ingresso).

Il portale dovrà interfacciarsi con il sistema centrale per mantenere aggiornata la situazione complessiva del parcheggio, evitando sovrapposizioni o doppie prenotazioni.

Inoltre, il sistema prevede agevolazioni per fasce d'età / occupazione e la possibilità di sottoscrivere un abbonamento mensile e annuale.

Requisiti funzionali e analisi dei casi d'uso

Lo schema UML dei casi d'uso riportato nella Figura 1 illustra i requisiti funzionali del sistema.

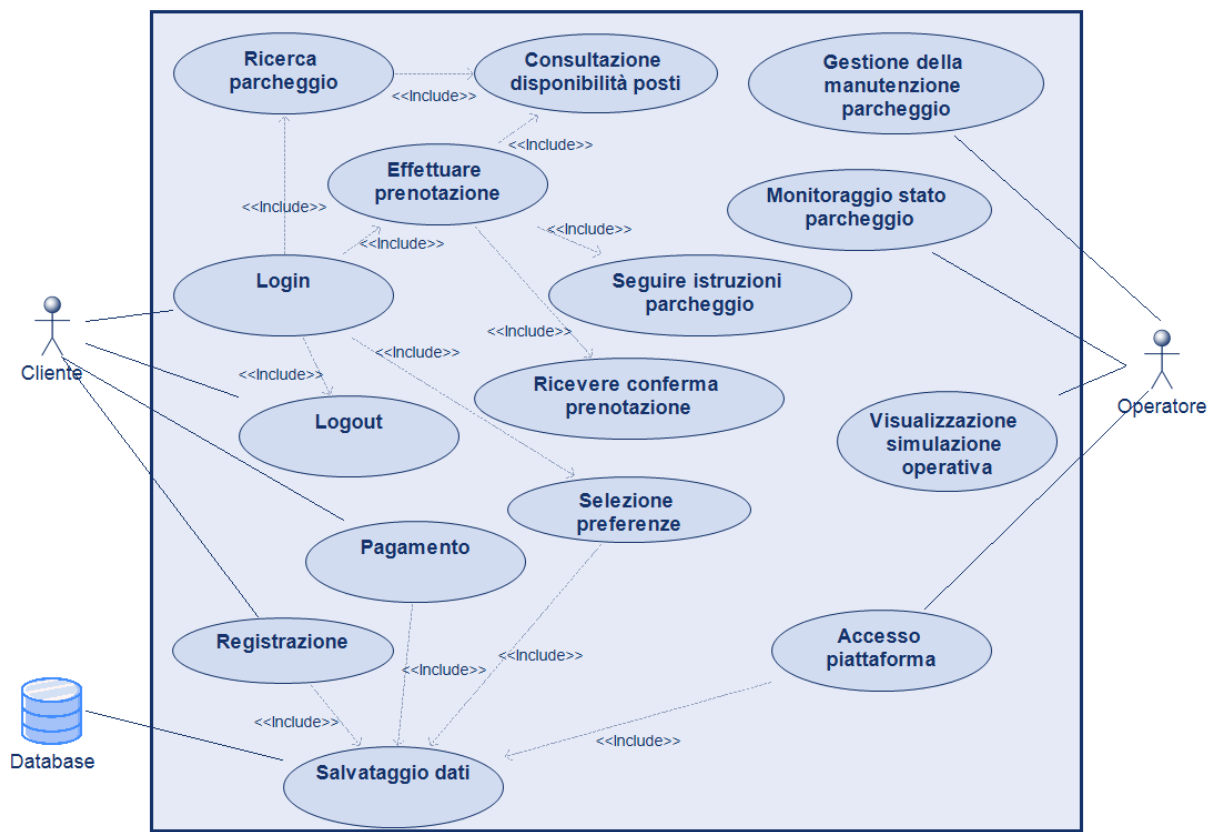


Figura 1: Diagramma UML dei casi d'uso

Durante la trattazione delle diverse iterazioni verranno implementati gradualmente al fine di raggiungere uno stato soddisfacente del progetto.

ID	Titolo
UC1.1	Login utente
UC1.2	Registrazione utente
UC1.3	Selezione preferenze
UC1.4	Logout
UC2.1	Login operatore
UC3.1	Salvataggio dati

Tabella 1: Casi d'uso iterazione 1

ID	Titolo
UC4.1	Ricerca parcheggio
UC4.2	Consultazione disponibilità posti
UC4.3	Effettua prenotazione
UC4.4	Ricevere consegna prenotazione
UC5.1	Monitoraggio stato parcheggio
UC5.2	Gestione della manutenzione parcheggio

Tabella 2: Casi d'uso iterazione 2

ID	Titolo
UC6.1	Seguire istruzioni parcheggio
UC7.1	Visualizzazione simulazione operativa

Tabella 3: Casi d'uso iterazione 3

Topologia del sistema

La topologia del sistema è di tipo client-server multilivello, con un sistema centrale che funge da nucleo dell'intero parcheggio multipiano. È presente un server applicativo collegato a un database che gestisce utenti, operatori, posti auto, ticket, prenotazioni e pagamenti.

Al server si connettono due tipologie di client: una componente Desktop utilizzata dagli operatori di cassa e di controllo, e una componente Web accessibile dagli utenti finali.

Il sistema centrale comunica inoltre con i dispositivi fisici del parcheggio tramite interfacce dedicate, consentendo il monitoraggio e il controllo in tempo reale dello stato del parcheggio. Tutte le operazioni critiche vengono elaborate sul server per garantire coerenza dei dati, sicurezza e prevenzione di conflitti come le doppie prenotazioni.

Design Pattern

Il sistema e i suoi componenti sono sviluppati utilizzando il design pattern architetturale Client-Server a più livelli, la comunicazione avviene tramite REST API.

Riconosciamo anche l'impiego del design pattern MVC che vede il client come "View", le API come "Controller" e l'unione di server e database come "Model".

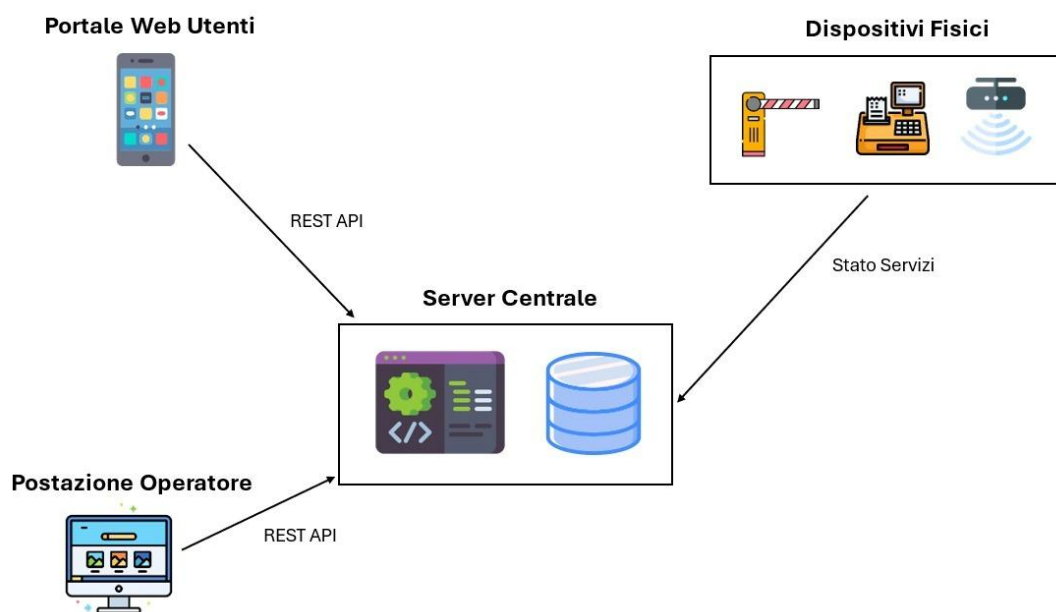


Figura 2: Topologia del sistema

Toolchain e tecnologie

Tool / Tecnologie	Utilizzo
Eclipse	IDE per sviluppo back-end
Visual Studio Code	IDE per sviluppo front-end
Spring Boot	Java Framework per realizzazione back-end
Dart	Linguaggio di programmazione per realizzazione front-end
Modelio	Tool per realizzazione diagrammi UML
Git e Github	Software e piattaforma per versionamento e condivisione di codice e documentazione
Postman	Piattaforma per test dinamico e documentazione delle API REST
JAutoDoc	Plugin che automatizza l'aggiunta di commenti Javadoc e intestazioni ai file sorgente Java
SonarLint	Tool per analisi statica
JUnit 5	Framework per Unit Testing
GitHub Projects Backlog	Collezione dinamica di task, storie utente, bug e idee

Tabella 4: Toolchain e tecnologie

Iterazione 1

Introduzione

- UC1 Utente
 - UC1.1: Registrazione
 - UC1.2: Login
 - UC1.3: Selezione preferenze
 - UC1.4: Logout
- UC2 Operatore
 - UC2.1: Accesso alla piattaforma
- UC3 Database
 - UC3.1: Salvataggio dati

UC1.1: Registrazione

Breve descrizione: Utente accede all'app per la prima volta, seleziona tasto per la registrazione, compila i campi richiesti e i dati vengono inviati al database

Attori coinvolti: Utente e Database

Trigger: Registrazione avvenuta con successo

Postcondizione: Dopo la registrazione l'utente viene rimandato alla schermata di login così da poter validare le credenziali appena create e accedere alla piattaforma

Procedimento:

- 1) Accesso all'app
- 2) Selezione bottone registrazione
- 3) Compilazione campi
- 4) Click sul tasto di invio
- 5) Reindirizzamento alla schermata di login

UC1.2: Login

Breve descrizione: Utente arriva sulla pagina di login dopo aver completato la registrazione e immette le sue credenziali così da accedere all'app e poter navigare; se non è il suo primo accesso l'utente viene automaticamente autenticato nella homepage

Attori coinvolti: Utente

Trigger: Login avvenuto con successo

Postcondizione: Dopo il login l'utente può accedere alla piattaforma e cercare / controllare i parcheggi

Procedimento:

- 1) Utente compila i campi di login
 - a. Entra nell'applicazione
- 1) Utente entrando nella piattaforma si trova direttamente nella homepage

UC1.3: Selezione preferenze

Breve descrizione: Dopo aver eseguito l'accesso nella schermata preposta al profilo personale, l'utente può selezionare delle preferenze relative al parcheggio che agevoleranno la ricerca delle strutture in tempo di ricerca

Attori coinvolti: Utente e Database

Trigger: Preferenze salvate correttamente

Postcondizione: Dopo la selezione l'utente trova solo i parcheggi che matchano con le preferenze che ha selezionato

Procedimento:

- 1) Utente clicca sul suo profilo personale
- 2) Seleziona l'impostazione delle preferenze
- 3) Sceglie quelle che più gli corrispondono
- 4) Salva le impostazioni

UC1.4: Logout

Breve descrizione: Utente loggato nella piattaforma clicca sul tasto in alto a destra per seguire il logout dall'app.

Attori coinvolti: Utente

Trigger: Logout avvenuto con successo

Postcondizione: Dopo il logout l'utente può riaccedere alla piattaforma o uscire da quest'ultima

Procedimento:

- 1) Utente seleziona icona in alto a destra
- 2) Conferma di voler uscire dalla piattaforma

UC2.1: Accesso alla piattaforma

Breve descrizione: L'operatore può accedere alla piattaforma inserendo le credenziali che gli vengono date e il nome della struttura in cui lavora

Attori coinvolti: Operatore e Database

Trigger: Accesso avvenuto con successo

Postcondizione: Dopo l'accesso, l'operatore può controllare che nel parcheggio tutto funzioni correttamente

Procedimento:

- 1) Operatore accede alla piattaforma
- 2) Controlla che tutto funzioni nel parcheggio

UC3.1: Salvataggio dati

Breve descrizione: I dati relativi alla registrazione degli utenti, ai pagamenti, agli accessi e alle uscite dal parcheggio vengono salvate nel Database

Attori coinvolti: Utente, Operatore e Database

Trigger: Registrazione avvenuta con successo/Pagamento avvenuto con successo/Ingresso/Uscita

Postcondizione: I dati vengono registrati nel Database per conservare tutte le operazioni e per effettuare a posteriori analisi sui dati (per esempio: previsioni sull'anno successivo)

Procedimento:

- 1) Registrazione utente/Pagamento/Ingresso/Uscita
- 2) Salvataggio dati

Diagramma delle componenti UML

Una volta scelti i casi d'uso per l'iterazione e definite le caratteristiche di design, è stata formalizzata una prima architettura software del nostro sistema.

I casi d'uso sono stati raggruppati e per ognuno è stato introdotto un componente *boundary*, *control* o *data*.

- <<*boundary*>> - componente front-end con cui gli attori si interfacciano;
- <<*controller*>> - componente back-end che gestisce la logica del programma ed espone le API;
- <<*service*>> - componente preposto alla logica applicativa che coordina le operazioni e decide "cosa fare";
- <<*repository*>> - componente di astrazione che incapsula query e operazioni CRUD verso il database;
- <<*database*>> - componente back-end che interagisce col database.

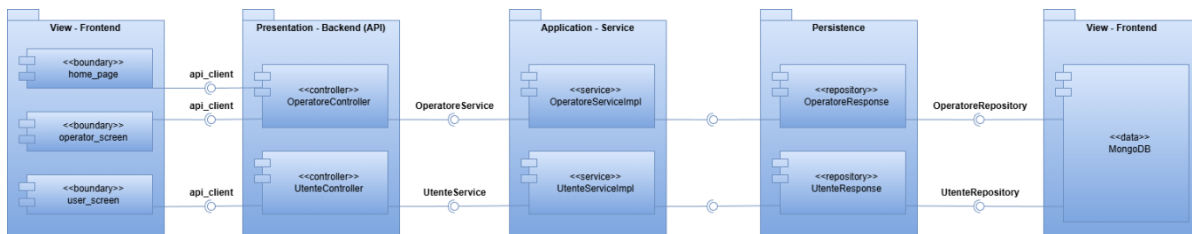


Figura 3: Diagramma delle componenti UML

Diagramma delle classi per le interfacce

Il diagramma rappresentante le interfacce del sistema con le relative funzioni e i dati I/O.

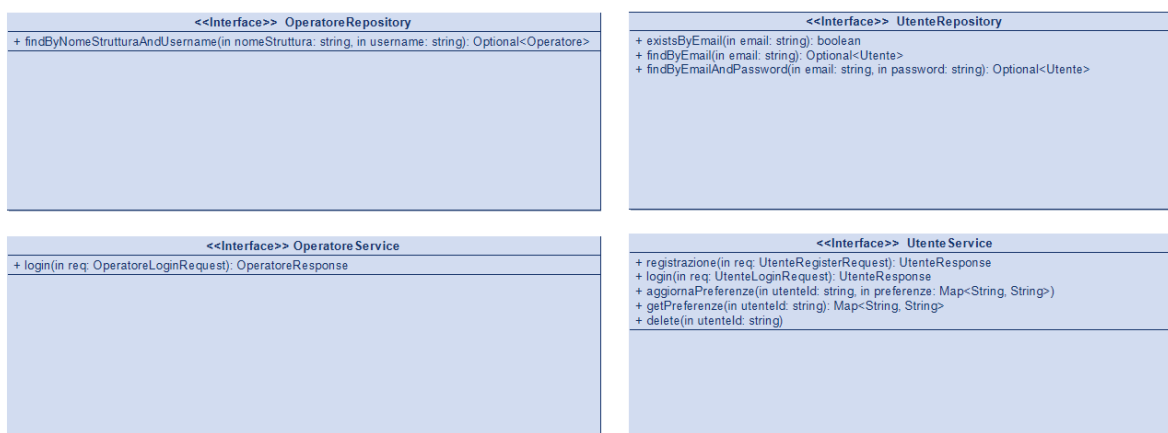


Figura 4: Diagramma delle classi UML per interfacce

Diagramma delle classi per tipi di dato

Diagramma con i tipi di dato necessari allo sviluppo dell'applicazione.

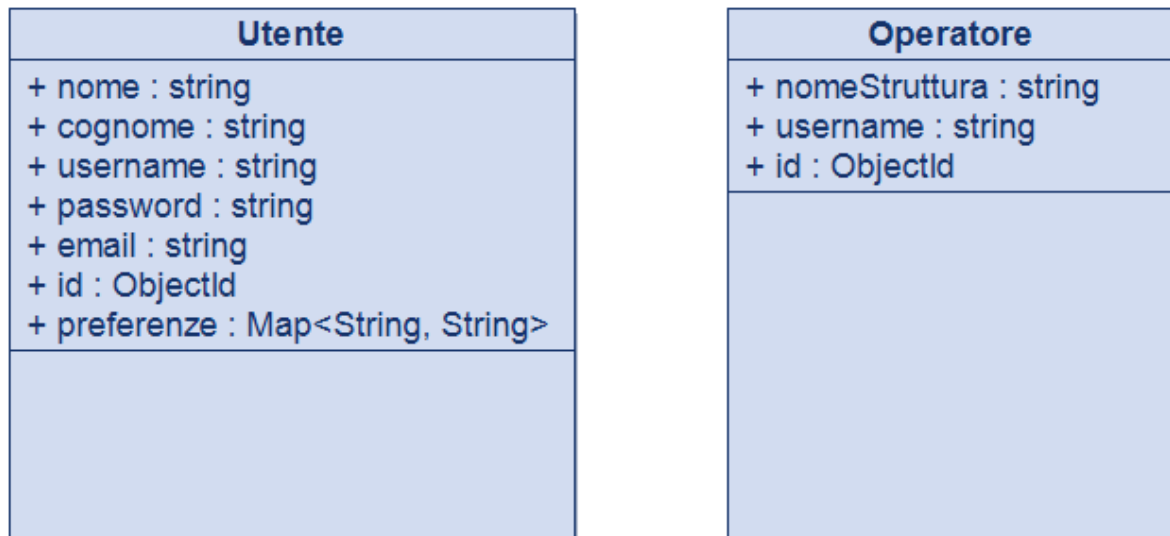


Figura 5: Diagramma delle classi UML per tipi di dato

Diagramma di deployment

Mediante l'utilizzo di un Deployment Diagram UML viene rappresentata la gestione hardware e software del sistema, di seguito sono presentati i componenti nei seguenti nodi:

- *TelefonoUtente*, nodo su cui un utente può registrarsi e loggarsi alla piattaforma
- *PortaleOperatore*, nodo su cui un operatore può loggarsi nella piattaforma per avere una panoramica dei servizi;
- *WebServer*, espone le API richieste lato front-end;
- *MongoDB*, si occupa dello storage dei dati.

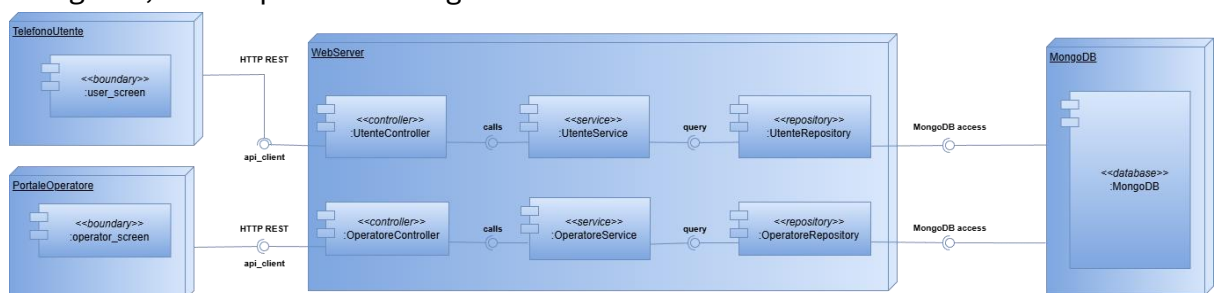


Figura 6: Diagramma di deployment UML

Testing

Analisi statica

Per garantire l'alta qualità, la manutenibilità e la sicurezza del codice sviluppato durante l'iterazione 1 del progetto, è stata eseguita un'analisi statica integrata.

Il tool scelto è SonarLint (plugin di SonarQube), utilizzato in Eclipse, durante la procedura ha identificato e guidato la risoluzione di Code Smells, potenziali Bug e violazioni delle best practice di programmazione.

Analisi dinamica

L'analisi dinamica è stata effettuata tramite:

- JUnit 5 per l'esecuzione dei test;
- Interazione reale con MongoDB, i comportamenti riflettono il funzionamento effettivo del sistema;
- Inserimento e rimozione temporanea dei dati, ogni test genera dati unici per evitare collisioni tra esecuzioni.

Questa metodologia garantisce risultati ripetibili e validi anche in presenza di test multipli.

Componenti Coinvolte

1. Modulo Utente
2. Modulo Operatore
3. Modulo Connessione

Struttura dei Test Dinamici

Sono stati realizzati test JUnit 5 sulle seguenti classi:

1. ConnessioneTest
2. UtenteTest
3. DatiUtentiTest
4. OperatoreTest
5. DatiOperatoriTest

Risultati dell'Analisi Dinamica

Tutti i test progettati hanno confermato che:

- il sistema interagisce correttamente con il database
- la logica applicativa rispetta il comportamento atteso
- eccezioni e condizioni di errore sono correttamente rilevate
- la separazione tra interfacce e implementazioni è coerente
- i metodi critici (loginDB, registrazioneDB, esisteOperatore) reagiscono correttamente sia a input validi che non validi

Non sono emersi malfunzionamenti critici.

Documentazione API

Queste sono le API sviluppate nella prima iterazione, è possibile prenderne visione tramite la collezione Postman presente nel repository GitHub:

- API per la registrazione e il login dell'utente
- API per la selezione delle preferenze dell'utente
- API per l'accesso dell'operatore

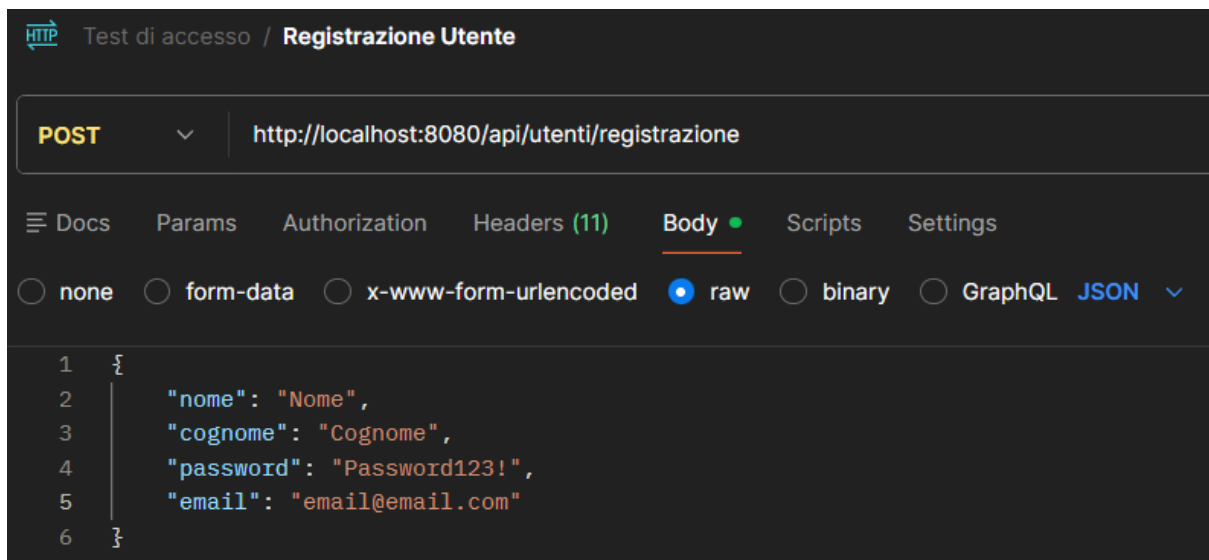


Figura 7: API per la registrazione dell'utente

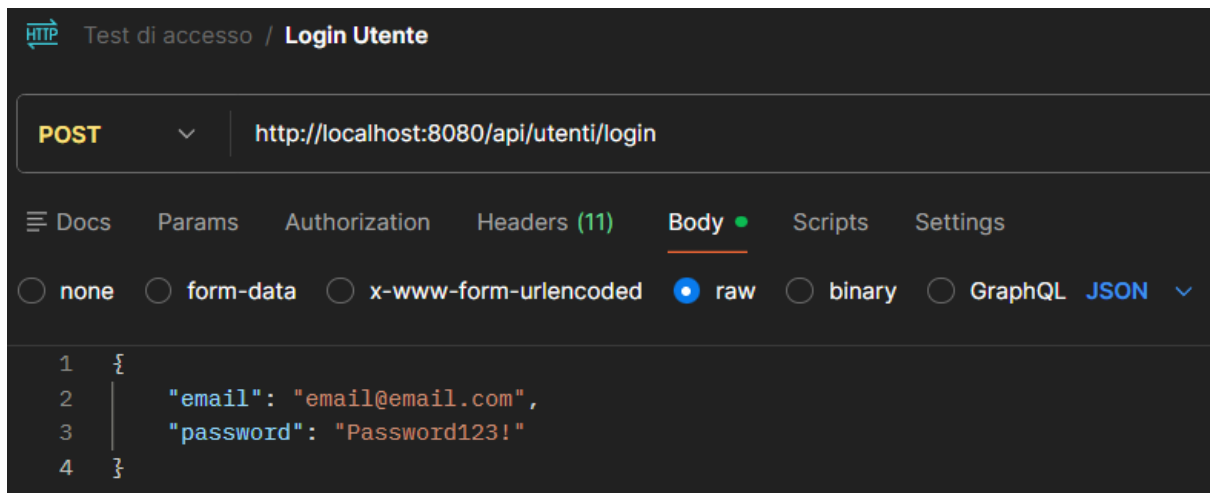


Figura 8: API per il login dell'utente

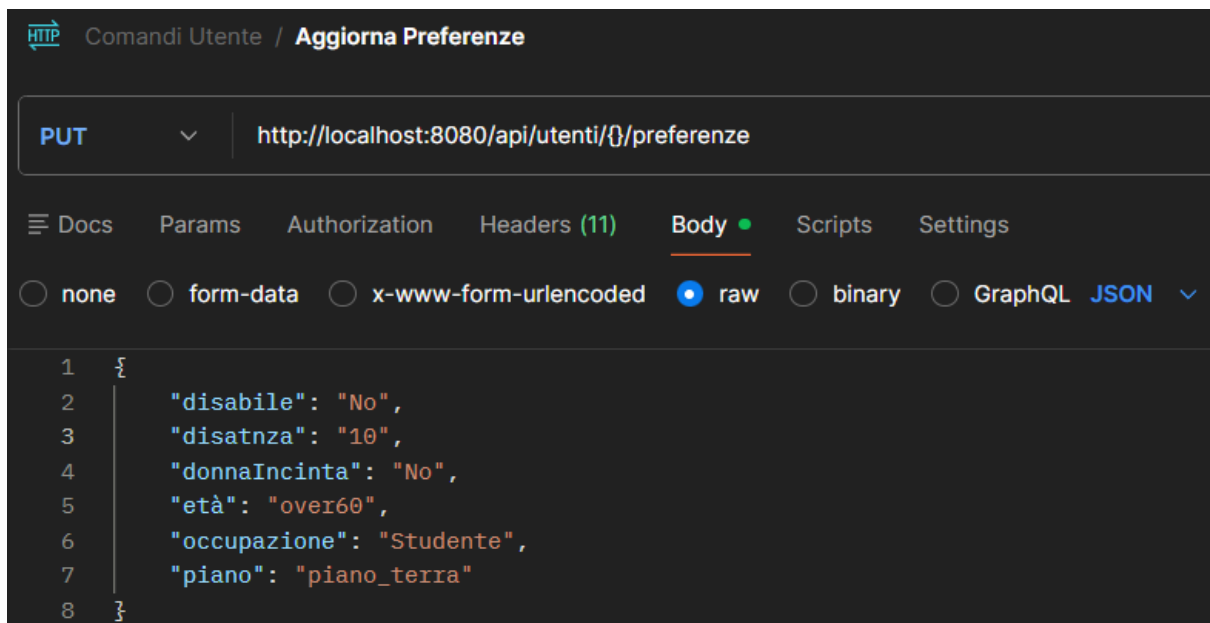


Figura 9: API per aggiornare le preferenze

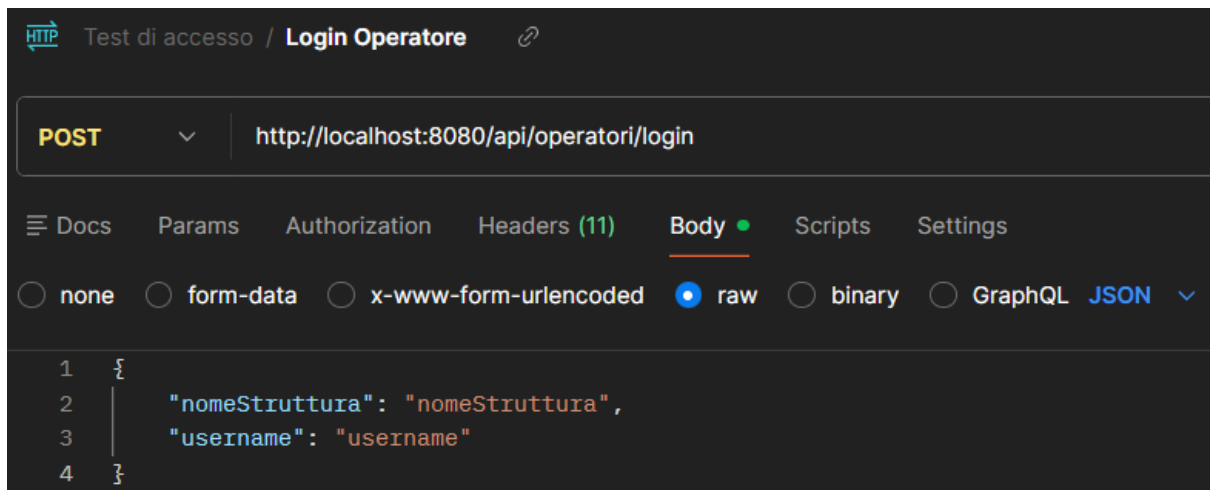


Figura 10: API per il login dell'operatore

Iterazione 2

Introduzione

- UC4 Prenotazione utente
 - UC4.1: Ricerca parcheggio
 - UC4.2: Consultazione disponibilità posti
 - UC4.3: Effettuare prenotazione
 - UC4.4: Ricevere conferma prenotazione
 - UC4.5: Consultazione storico prenotazioni
- UC5 Analitiche parcheggio
 - UC5.1: Monitoraggio stato parcheggio
 - UC5.2: Gestione della manutenzione parcheggio

UC4.1: Ricerca parcheggio

Breve descrizione: Utente cerca, nella sezione dell'app apposita, i parcheggi presenti nel database collegati agli operatori esistenti

Attori coinvolti: Utente e Database

Trigger: La ricerca ha prodotto dei risultati

Postcondizione: Dopo la ricerca l'utente seleziona il parcheggio, tra quelli proposti, a lui più conveniente

Procedimento:

- 1) Selezione dell'area di ricerca
- 2) Click sulla barra di ricerca
- 3) Digitazione query
- 4) Presa visione dei risultati
- 5) Selezione della scelta

UC4.2: Consultazione disponibilità posti

Breve descrizione: Dopo aver eseguito la ricerca dei parcheggi l'utente vede accanto ad ognuno la disponibilità dei posti totali

Attori coinvolti: Utente e Database

Trigger: Corretta visualizzazione dei dati dei parcheggi

Postcondizione: Dopo aver preso visione dei posti disponibili l'utente seleziona il parcheggio a lui più congeniale

Procedimento:

- 1) Utente cerca i parcheggi

- 2) Prende visione della disponibilità dei singoli luoghi

UC4.3: Effettuare prenotazione

Breve descrizione: Utente una volta selezionato il parcheggio scelto, decide di effettuare una prenotazione presso quel luogo

Attori coinvolti: Utente e Database

Trigger: Prenotazione avvenuta con successo

Postcondizione: Dopo aver effettuato la prenotazione, l'utente si dirigerà presso il parcheggio all'orario stabilito e potrà accedervi

Procedimento:

- 1) Utente cerca parcheggi
- 2) Vede la disponibilità
- 3) Sceglie la struttura
- 4) Effettua prenotazione

UC4.4: Ricevere consegna prenotazione

Breve descrizione: Utente dopo aver prenotato il posto auto riceve un QR Code della prenotazione

Attori coinvolti: Utente e Database

Trigger: Ricezione codice avvenuto con successo

Postcondizione: Una volta arrivato alla struttura l'utente presenterà il QR Code alla colonnina di controllo

Procedimento:

- 1) Utente effettua prenotazione
- 2) Riceve codice di autenticazione

UC4.5: Consultazione dello storico prenotazioni

Breve descrizione: L'utente accede a una sezione dedicata dell'applicazione per visualizzare l'elenco completo di tutte le prenotazioni effettuate in precedenza

Attori coinvolti: Utente e Database

Trigger: L'utente seleziona la voce relativa allo storico dal menu a tendina dell'app

Postcondizione: L'utente prende visione dei dettagli delle proprie prenotazioni passate (ID, orario, parcheggio e QR code)

Procedimento:

- 1) L'utente apre il menu a tendina dell'interfaccia principale
- 2) Selezione della voce "Le mie prenotazioni"
- 3) Il sistema richiede i dati al database tramite l'ID utente
- 4) Presa visione della lista dei risultati ottenuti

UC5.1: Monitoraggio stato parcheggio

Breve descrizione: L'operatore può accedere alla piattaforma inserendo le credenziali che gli vengono date e il nome della struttura in cui lavora

Attori coinvolti: Operatore e Database

Trigger: Accesso avvenuto con successo

Postcondizione: Dopo l'accesso, l'operatore può controllare che nel parcheggio tutto funzioni correttamente

Procedimento:

- 1) Operatore accede alla piattaforma
- 2) Controlla che tutto funzioni nel parcheggio

UC5.2: Gestione della manutenzione parcheggio

Breve descrizione: L'operatore può accedere alla piattaforma inserendo le credenziali che gli vengono date e il nome della struttura in cui lavora

Attori coinvolti: Operatore e Database

Trigger: Accesso avvenuto con successo

Postcondizione: Dopo l'accesso, l'operatore può controllare che nel parcheggio tutto funzioni correttamente

Procedimento:

- 1) Operatore accede alla piattaforma
- 2) Controlla che tutto funzioni nel parcheggio

Diagramma delle componenti UML

I casi d'uso scelti per la seconda iterazione sono stati aggiunti al component diagram della prima iterazione.

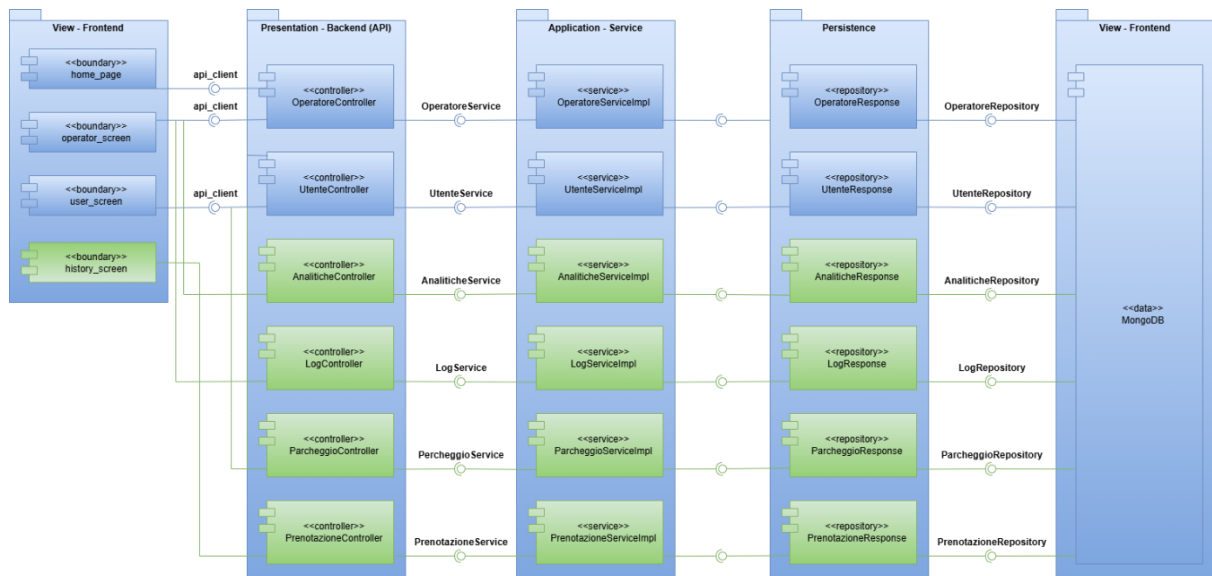


Figura 11: Diagramma delle componenti UML

Diagramma delle classi per le interfacce

Il diagramma rappresentante le interfacce dei nuovi casi d'uso introdotti.



Figura 12: Diagramma delle classi UML per interfacce

Diagramma delle classi per tipi di dato

I tipi di dato Analitiche, Log, Parcheggio e Prenotazione vengono inseriti nel Diagramma per tipo di dato dell'iterazione precedente, il nuovo diagramma dispone le nuove interazioni tra i dati.

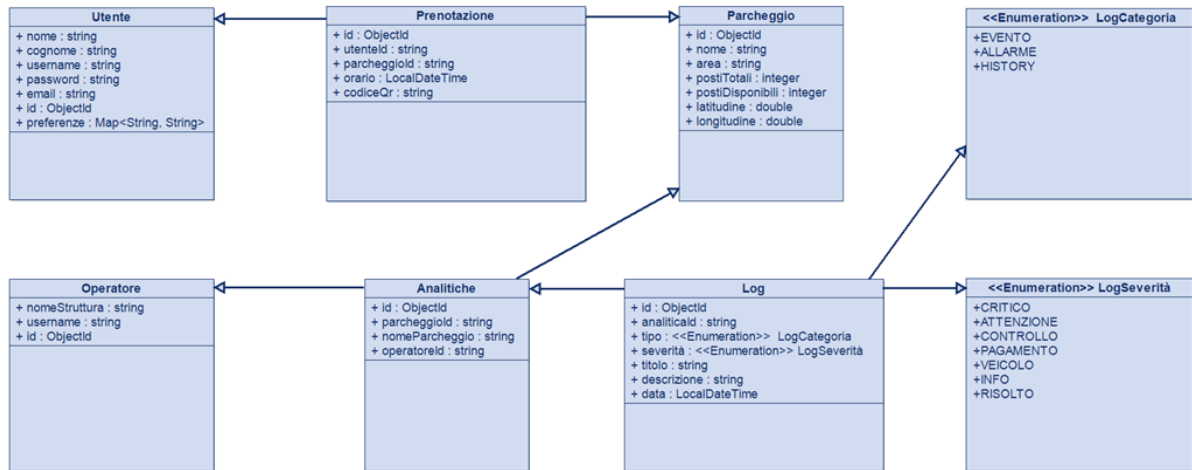


Figura 13: Diagramma delle classi UML per tipi di dato

Diagramma di deployment

I nuovi componenti definiti in questa seconda iterazione sono stati inseriti nel Deployment Diagram dell'iterazione numero uno.

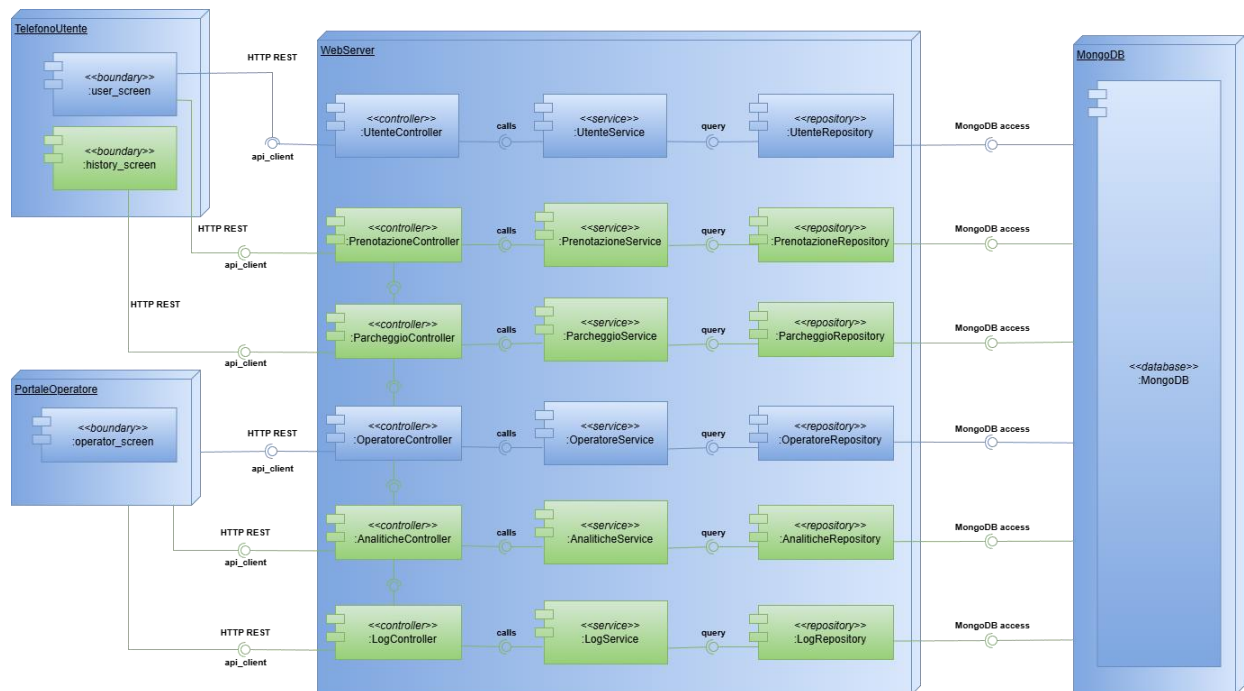


Figura 14: Diagramma di deployment UML

Testing

Analisi statica

Per garantire l'alta qualità, la manutenibilità e la sicurezza del codice sviluppato durante l'iterazione 2 del progetto, è stata eseguita un'analisi statica integrata.

SonarLint ha identificato e guidato la risoluzione di *Code Smells*, potenziali *Bug* e violazioni delle *best practice* di programmazione.

Analisi dinamica

Per l'iterazione 2 sono stati effettuati i test delle seguenti funzioni:

- Creazione, recupero e salvataggio delle Analitiche
- Creazione, recupero, salvataggio e aggiornamento dei Log
- Ricerca e prenotazione di un Parcheggio
- Recupero storico delle Prenotazioni

Struttura dei Test Dinamici

Sono stati realizzati test JUnit 5 sulle seguenti classi:

1) AnaliticheController

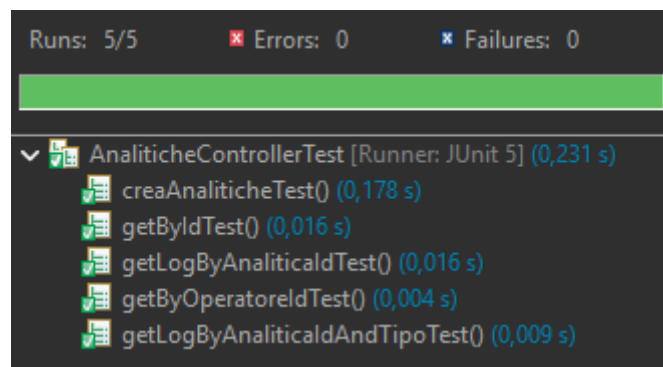


Figura 15: Esito test JUnit sulla classe AnaliticheController

2) AnaliticheServiceImpl

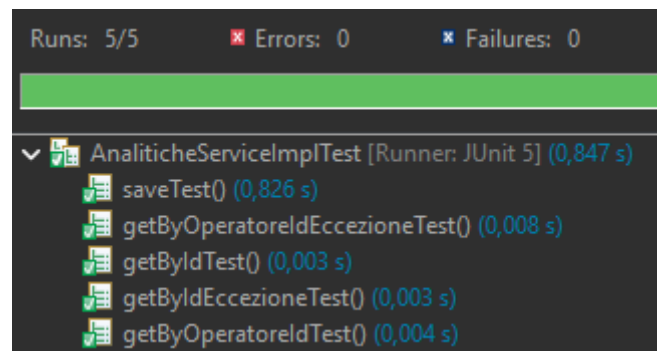


Figura 16: Esito test JUnit sulla classe AnaliticheServiceImpl

3) LogController

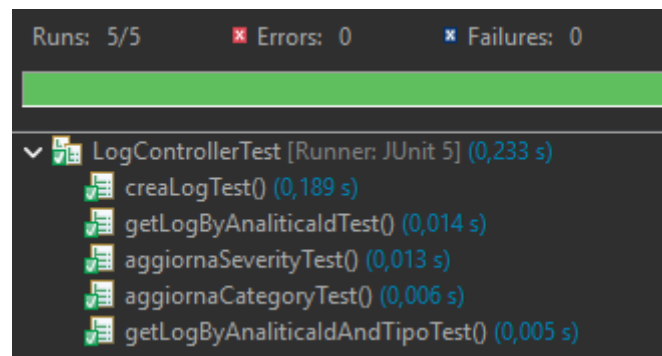


Figura 17: Esito test JUnit sulla classe LogController

4) LogServiceImpl

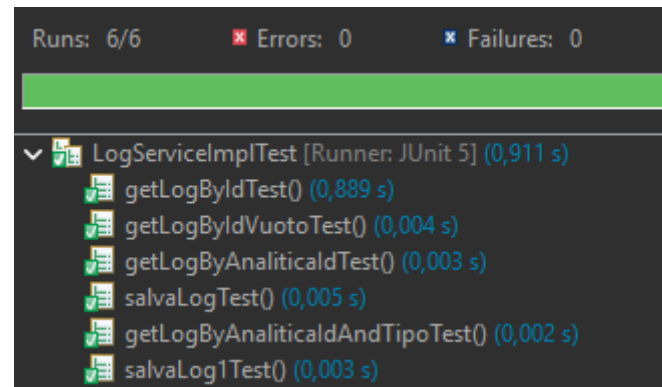


Figura 18: Esito test JUnit sulla classe LogServiceImpl

5) ParcheggioController

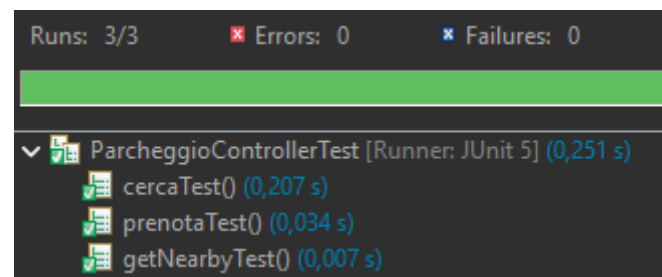


Figura 19: Esito test JUnit sulla classe ParcheggioController

6) ParcheggioServiceImpl

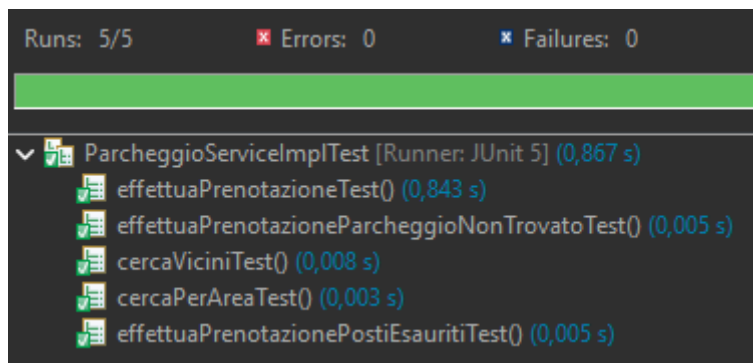


Figura 20: Esito test JUnit sulla classe ParcheggioServiceImpl

7) PrenotazioneController

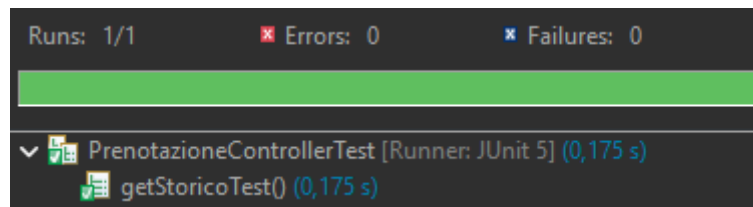


Figura 21: Esito test JUnit sulla classe PrenotazioneController

8) PrenotazioneServiceImpl

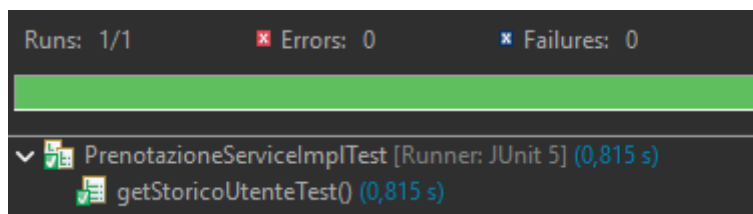


Figura 22: Esito test JUnit sulla classe PrenotazioneServiceImpl

Coverage

PMG-backend	94,5 %	4.631	272	4.903
> src/main/java	88,0 %	1.619	221	1.840
> src/test/java	98,3 %	3.012	51	3.063

Figura 23: Coverage fornito dai test JUnit del progetto

Documentazione API

Queste sono le API sviluppate nella prima iterazione, è possibile prenderne visione tramite la collezione Postman presente nel repository GitHub:

- API per la ricerca dei parcheggi in base all'area

- API per la prenotazione di un posto nel parcheggio
- API per recuperare lo storico delle prenotazioni dell'utente
- API per recuperare le analitiche del parcheggio
- API per recuperare i log di un'analitica

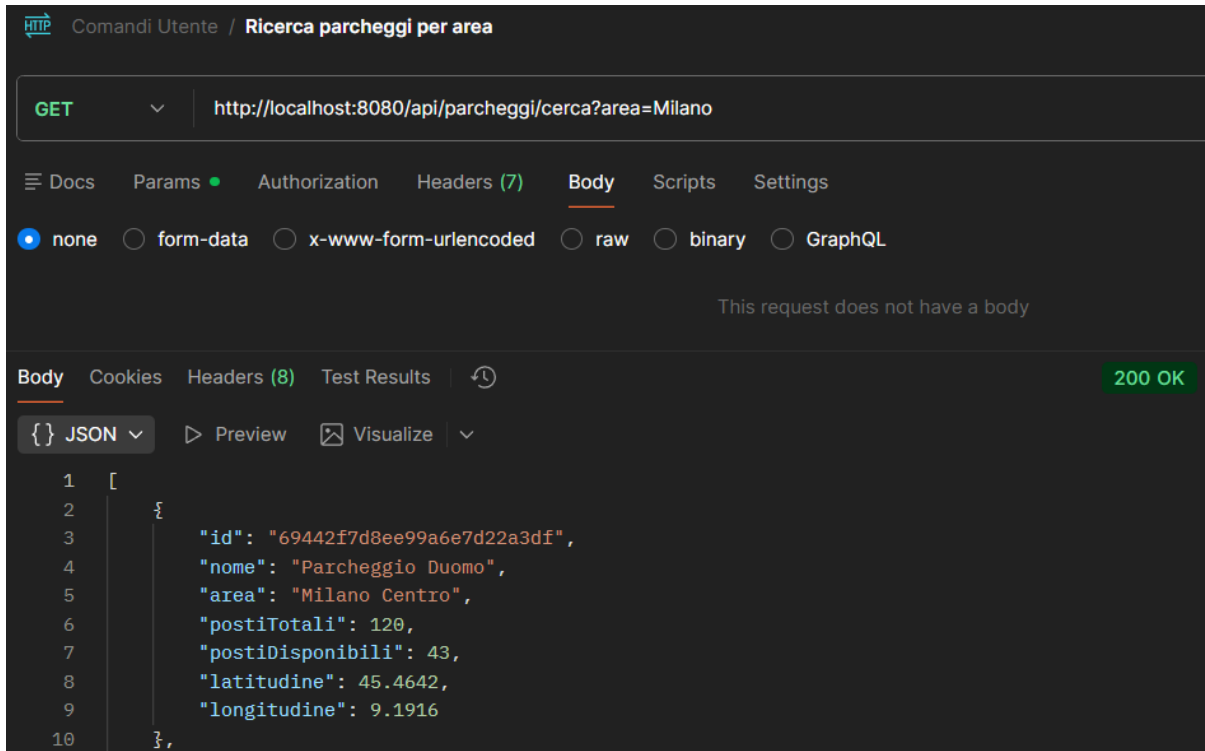


Figura 24: API per la ricerca dei parcheggi in base all'area

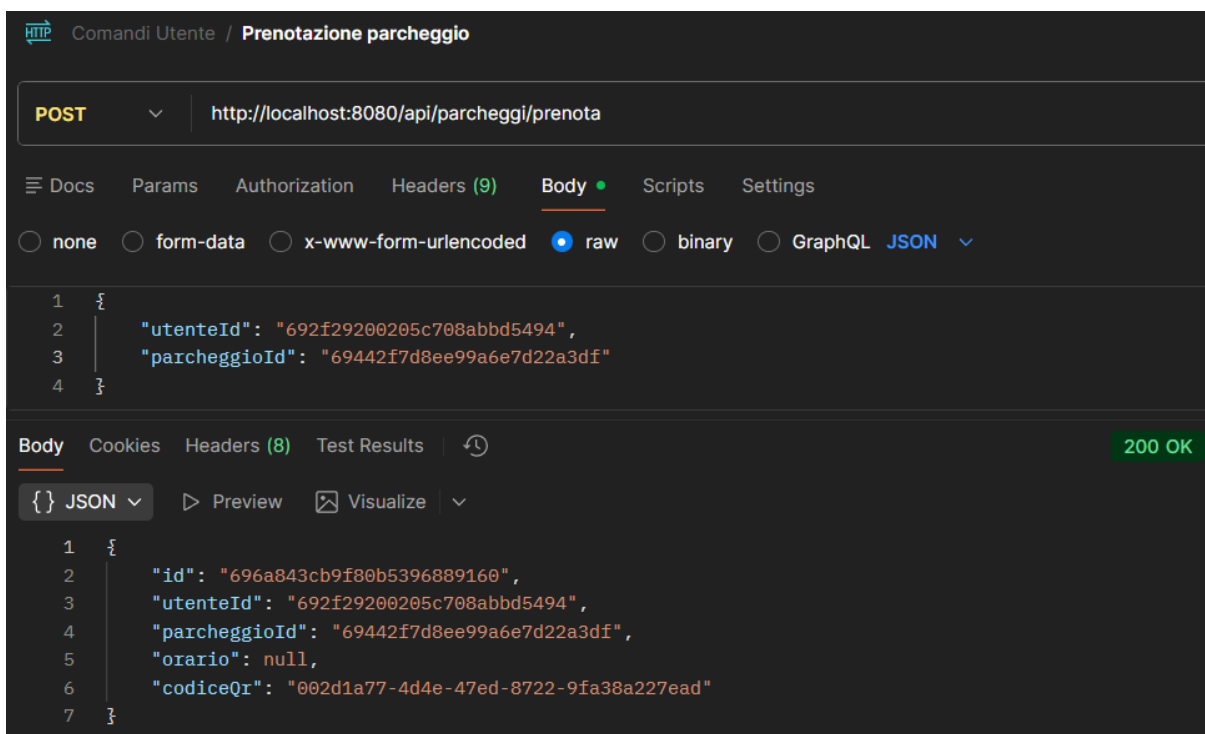


Figura 25: API per la prenotazione di un posto nel parcheggio

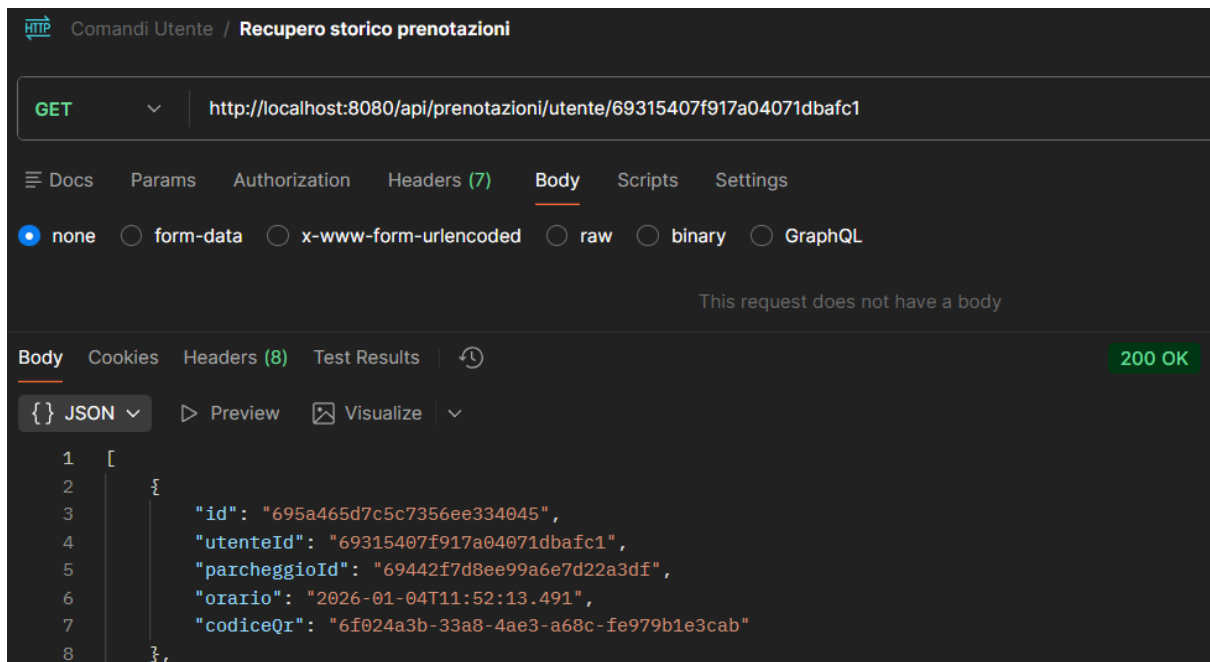


Figura 26: API per recuperare lo storico delle prenotazioni dell'utente

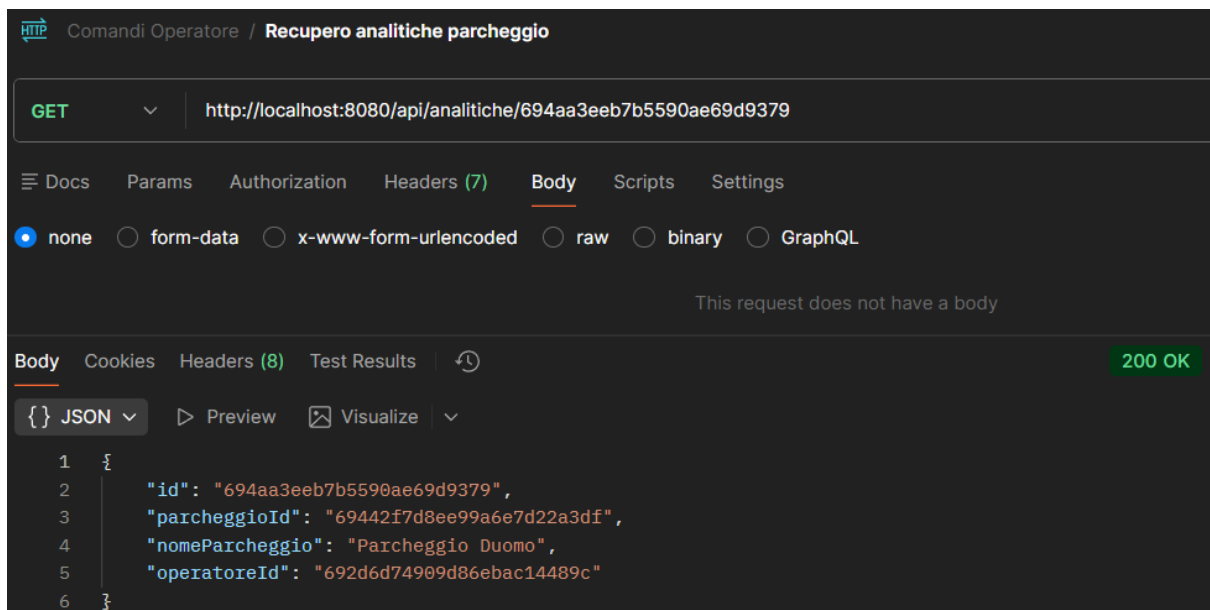


Figura 27: API per recuperare le analitiche del parcheggio

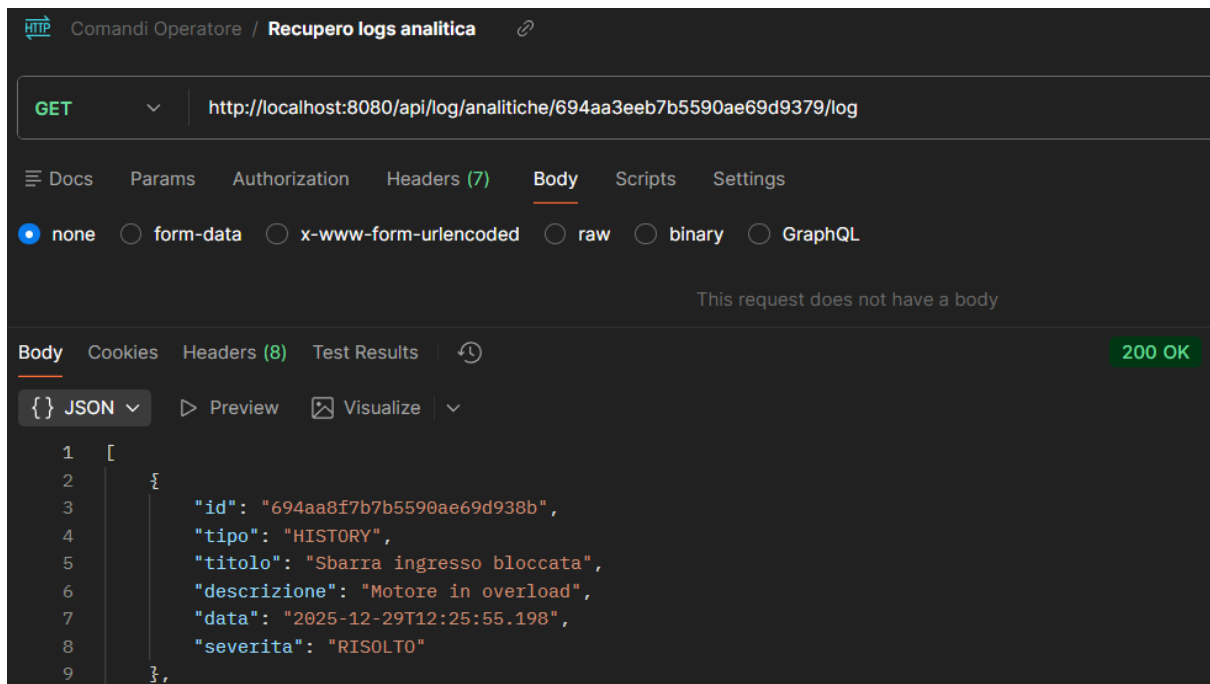


Figura 28: API per recuperare i log di un'analitica

Iterazione 3

Introduzione

- UC6: Navigazione utente
 - UC 6.1: Seguire istruzione per arrivare al parcheggio
 - UC 6.2: Cancellazione prenotazione
- UC7: Monitoraggio operatore
 - UC 7.1: Visualizzazione simulazione operativa
 - UC 7.2: Validazione QR Code e Accesso
 - UC 7.3: Consultazione Cronologia Giornaliera
- UC 8 Interventi manuali
 - UC 8.1: Modifica Manuale Disponibilità
 - UC 8.2: Blocco Emergenza Parcheggio
- UC9: Algoritmo di ottimizzazione selezione parcheggio
- UC10: Algoritmo di calcolo tariffa prenotazione
- UC11: Algoritmo di identificazione percorso migliore BFS

UC 6.1: Seguire istruzione per arrivare al parcheggio

Breve descrizione: Utente cerca, nella sezione dell'app apposita, i parcheggi in base alla zona dove vuole parcheggiare.

Attori coinvolti: Utente, Database

Trigger: La ricerca mostra i parcheggi disponibili e non nella mappa

Postcondizione: Dopo la ricerca l'utente seleziona il parcheggio, tra quelli proposti, a lui più conveniente.

Procedimento:

- 1) Click sulla barra di ricerca
- 2) Digitazione query
- 3) Presa visione dei risultati
- 4) Selezione della scelta

UC 6.2: Gestione prenotazione

Breve descrizione: Utente va nella sezione dell'app apposita e può gestire la sua prenotazione.

Attori coinvolti: Utente, Database

Trigger: La sezione mostra le prenotazioni

Postcondizione: Dopo aver selezionato la prenotazione, questa mostra il qr code oppure può essere eliminata

Procedimento:

- 1) Click sulla rotellina delle impostazioni
- 2) Click su "le mie prenotazioni"
- 3) Click sulla prenotazione

UC 7.1: Visualizzazione simulazione operativa

Breve descrizione: Operatore dopo aver eseguito il login, ha la possibilità di vedere tutti gli allarmi e gli eventi sia presenti sia passati relativi alla sua struttura e, spostandosi nella dashboard del parcheggio fisico visualizzare tutti i posti occupati, disabilitati e liberi

Attori coinvolti: Operatore, Database

Trigger: Login avvenuto con successo

Postcondizione: Dopo il login l'operatore può accedere alla piattaforma e controllare la propria struttura

Procedimento:

- 1) Operatore compila i campi di login
- 2) Visualizza sezione eventi
- 3) Clicca sulla sezione parcheggio
- 4) Visualizzazione reale della struttura

UC 7.2: Validazione QR Code e Accesso

Breve descrizione: L'operatore (o il sistema di controllo accessi) effettua la scansione del QR Code presentato dall'utente per validare la prenotazione, consentire l'accesso fisico al parcheggio e far partire il conteggio del tempo per il pagamento finale.

Attori coinvolti: Operatore, Database, Utente.

Trigger: L'utente arriva all'ingresso del parcheggio e mostra il codice QR.

Postcondizione: La prenotazione viene segnata come "inCorso", la sbarra si alza e il sistema registra l'orario di ingresso effettivo.

Procedimento:

- 1) Scansione del codice QR tramite dispositivo dell'operatore.
- 2) Invio del codice al backend per la verifica di validità e univocità.
- 3) Controllo del database: verifica che il codice esista e non sia già stato usato.
- 4) Aggiornamento dello stato della prenotazione in "In corso".

UC 7.3: Consultazione Cronologia Giornaliera

Breve descrizione: L'operatore consulta l'elenco di tutte le attività (prenotazioni effettuate, ingressi e uscite) avvenute all'interno di uno specifico parcheggio durante la giornata lavorativa.

Attori coinvolti: Operatore, Database.

Trigger: L'operatore accede alla sezione "Log Giornaliero" del pannello gestionale.

Postcondizione: L'operatore ottiene una lista dettagliata che permette di monitorare il flusso di veicoli e l'andamento degli incassi previsti.

Procedimento:

- 1) Selezione della data (default: data odierna).
- 2) Invio della richiesta di filtraggio al database.
- 3) Presa visione dell'elenco delle prenotazioni e degli accessi validati.
- 4) Visualizzazione del totale dei posti occupati rispetto a quelli prenotati.

UC 8.1: Modifica Manuale Disponibilità

Breve descrizione: L'operatore decide di creare un nuovo allarme per la sua struttura o, nella sezione parcheggio, seleziona un posto e lo disabilita

Attori coinvolti: Operatore, Database, Utenti (indirettamente).

Trigger: Situazione di possibile pericolo non gestita dal sistema automatico.

Postcondizione: L'operatore vede la nuova segnalazione creata o, nel secondo caso, il posto disabilitato non viene preso in considerazione quando viene fatta una nuova prenotazione

Procedimento:

- 1) Selezione del tasto per creare nuovo allarme
 - a. Compilazione campi
 - b. Creazione del nuovo allarme

- 1) Selezione del posto
 - a. Conferma dell'azione di disabilitazione.

UC 8.2: Blocco Emergenza Parcheggio

Breve descrizione: L'operatore attiva una procedura di emergenza che interrompe istantaneamente la possibilità di effettuare nuove prenotazioni, rendendo il parcheggio "Chiuso" o "Non Disponibile" nell'applicazione.

Attori coinvolti: Operatore, Database, Utenti (indirettamente).

Trigger: Situazione di pericolo, guasto grave alla sbarra d'ingresso, allagamento o saturazione improvvisa non gestita dal sistema automatico.

Postcondizione: Il parcheggio scompare dai risultati di ricerca o appare con lo stato "Chiuso", impedendo nuovi flussi in entrata.

Procedimento:

- 1) Identificazione del parcheggio in stato critico nella dashboard.
- 2) Click sul pulsante di "Emergenza"
- 3) Selezione del motivo del blocco (opzionale, per reportistica).
- 4) Conferma dell'azione di blocco totale.

UC9: Algoritmo di ottimizzazione selezione parcheggio

Breve descrizione: L'utente prenota il posto auto all'interno del parcheggio, l'algoritmo tiene conto delle preferenze precedentemente confermate dall'utilizzatore e assegna la miglior posizione che gli verrà riservata

Attori coinvolti: Sistema

Trigger: Utente effettua prenotazione nella struttura

Postcondizione: Prenotazione confermata o fallita

Passi dell'algoritmo:

- 1) Ricezione della richiesta di prenotazione, il sistema riceve una richiesta contenente:
 - a. ID utente;
 - b. Fascia oraria di utilizzo del parcheggio;
 - c. Preferenze dell'utente:
 - i. Condizione di disabilità (boolean)
 - ii. Gravidanza (boolean)
 - iii. Età
 - iv. Occupazione
 - v. Distanza massima desiderata dall'uscita
 - vi. Piano preferito del parcheggio
- 2) Validazione dei dati, il sistema controlla che:
 - a. I valori inseriti siano coerenti;
 - b. Le preferenze obbligatorie siano presenti;In caso di errore, l'algoritmo termina con esito negativo.
- 3) Filtraggio dei posti compatibili, dal database vengono selezionati solo i posti che rispettano i vincoli non negoziabili riducendo l'insieme dei candidati.
- 4) Inizializzazione delle variabili di supporto:
 - a. migliorPosto $\leftarrow$ NULL;
 - b. punteggioMassimo $\leftarrow -\infty$.

- 5) Valutazione dei posti candidati tramite funzione di punteggio, per ogni posto compatibile si calcola un punteggio di qualità basato sulle preferenze:
 - a. Se l'utente è disabile e il posto è riservato → punteggio alto;
 - b. Se l'utente è incinta → più il posto è vicino all'uscita, maggiore il punteggio;
 - c. Maggiore è la distanza reale dall'uscita rispetto a quella desiderata → penalità;
 - d. Se il piano del posto coincide con quello preferito → bonus;
 - e. Alcune categorie possono dare priorità ai piani più accessibili.
- 6) Aggiornamento della soluzione ottima, dopo aver calcolato il punteggio del posto corrente:
 - a. Se il punteggio è maggiore di `punteggioMassimo`, allora:
 - i. `migliorPosto ← postoCorrente`;
 - ii. `punteggioMassimo ← punteggioCorrente`.
- 7) Selezione del posto ottimale, al termine della valutazione di tutti i posti:
 - a. `migliorPosto` rappresenta il posto che massimizza la soddisfazione dell'utente.
- 8) Assegnazione e aggiornamento del database, il sistema:
 - a. Assegna formalmente il posto all'utente;
 - b. Aggiorna lo stato del posto come "prenotato";
 - c. Registra l'operazione per fini statistici e di tracciamento.
- 9) Restituzione del risultato all'utente, il quale riceve:
 - a. Conferma della prenotazione;
 - b. Identificativo del posto;
 - c. Piano e posizione.

Di seguito viene mostrato lo pseudocodice dell'algoritmo con l'analisi della complessità, nella prima parte dell'analisi vengono visti tutti i posti per verificare la disponibilità e i vincoli di rigidità imposti, successivamente invece vengono esaminati tali posti tenendo conto dell'idoneità, nel caso peggiore troviamo n candidati, nel caso migliore 1 candidato. In ultima istanza quindi la complessità dell'algoritmo è scritta nell'Equazione 1.

$$O(n) + O(n) = O(2n) = O(n)$$

Equazione 1: Calcolo della complessità algoritmo assegnazione posto ottimo

Alg SelezionePostoOttimale(utente, listaPosti)

// Output

`postoAssegnato` // ritorna il posto assegnato all'utente

// Filtraggio dei posti disponibili

`postiCandidati ← lista vuota`

// Complessità $O(n)$

for `posto` in `listaPosti`

 if `posto == disponibile`

```

        if utente.disabile == true && posto.riservatoDisabili == false
            scarta posto
        if utente.incinta == true && posto.riservatoIncinta == false
            scarta posto
        else
            aggiungi posto a postiCandidati
        end if
    end if
end for

// Inizializzazione
migliorPosto ← NULL
punteggioMassimo ←  $-\infty$ 

// Complessità  $O(k)$ , caso peggiore  $O(n)$ 

// Valutazione dei posti candidati
for posto in postiCandidati
    punteggio ← 0
    // Preferenza disabilità
    if utente.disabile == true && posto.riservatoDisabili == true
        punteggio ← punteggio + 50
    end if
    // Preferenza gravidanza
    if utente.incinta == true
        punteggio ← punteggio + (100 - posto.distanzaUscita)
    end if
    // Piano preferito
    if posto.piano == utente.pianoPreferito
        punteggio ← punteggio + 20
    end if
    // Distanza desiderata dall'uscita

```

```
    differenza ← abs(posto.distanzaUscita - utente.distanzaPreferita)

    punteggio ← punteggio - differenza

    // Aggiornamento del migliore
    if punteggio > punteggioMassimo
        punteggioMassimo ← punteggio
        migliorPosto ← posto
    end if

end for

// Assegna migliorPosto all'utente
postoAssegnato ← migliorPosto

return postoAssegnato

end Alg
```

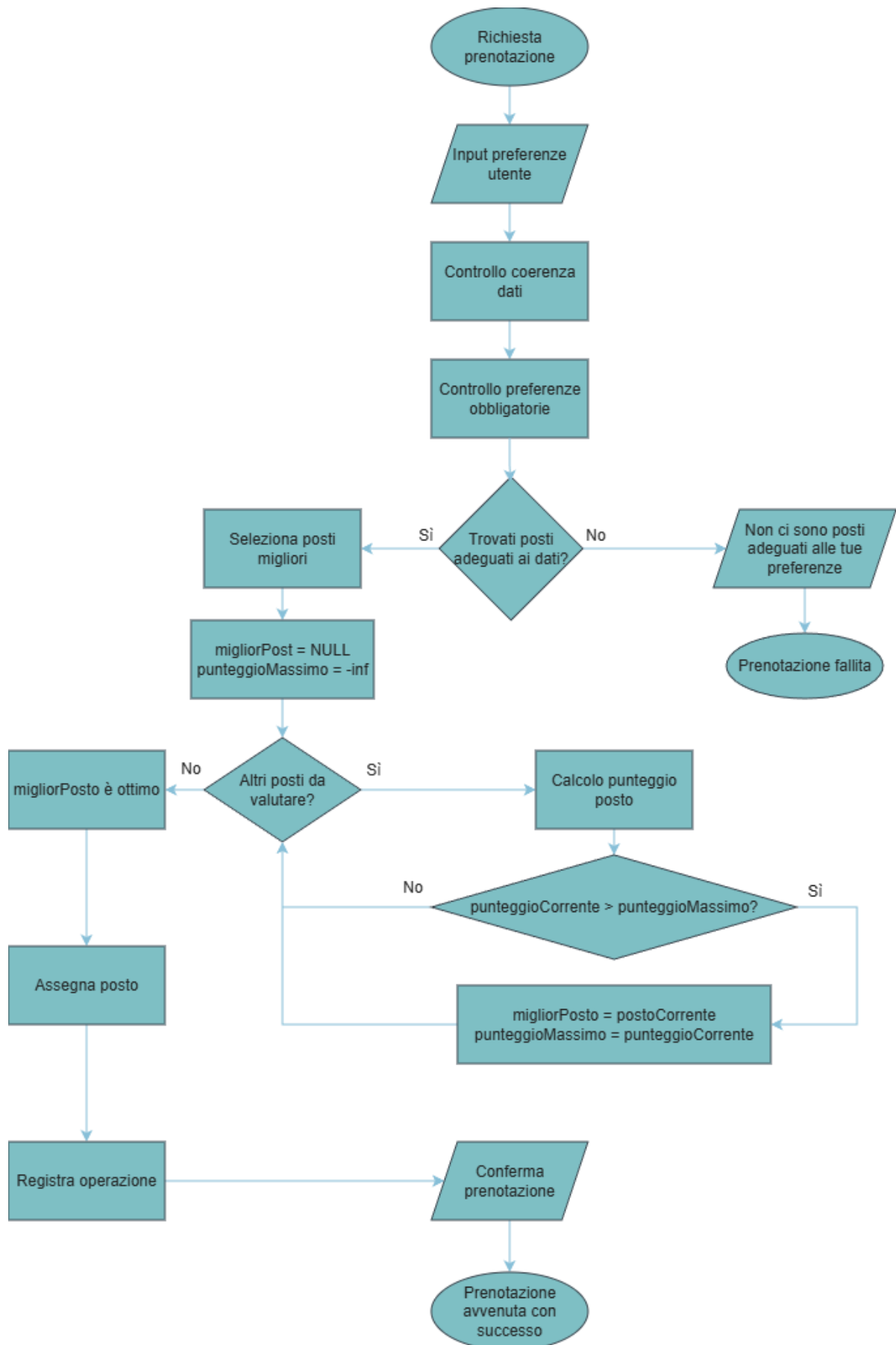


Diagramma 1: Flowchart dell'algoritmo assegnazione posto ottimo

UC10: Algoritmo di calcolo tariffa prenotazione

Quando clicchi su paga ti decurta 0.0 dal conto da quel momento partono i 10 minuti, se fai vedere QRcode prima dello scadere ti preleva l'importo normale sennò ti aggiunge una penale

Breve descrizione: L'algoritmo calcola il costo totale che l'utente dovrà pagare quando conclude la sua permanenza nel parcheggio, partendo da un costo base orario di 3 €/ora, restituisce l'importo finale da pagare, applicando:

- Sconti e maggiorazioni basati su preferenze dell'utente (età, occupazione)
- Maggiorazioni legate alla fascia oraria e al tipo di giorno
- Fee dinamica in base all'occupazione del parcheggio durante l'attesa
- Penale per ritardo nella convalida del QR code
- Sconti legati alla durata complessiva della permanenza

Attori coinvolti: Sistema

Trigger: Utente va a pagare il parcheggio

Postcondizione: Costo totale calcolato e pagato dall'utente

Passi dell'algoritmo:

- 1) Calcolo del costo base:
 - a. $\text{costoBase} \leftarrow \text{durataOre} \times 3$
- 2) Applicazione modificatori legati all'utente:
 - a. Età $\rightarrow$ sconto percentuale
 - b. Occupazione $\rightarrow$ sconto percentuale
- 3) Applicazione modificatori temporali:
 - a. Fascia oraria $\rightarrow$ maggiorazione percentuale
 - b. Tipo di giorno $\rightarrow$ maggiorazione percentuale
- 4) Calcolo della fee per attesa QR in base all'occupazione:
 - a. $< 50\% \rightarrow$ nessuna fee
 - b. $50-80\% \rightarrow 0.05 \text{ €/min}$
 - c. $80\% \rightarrow 0.10 \text{ €/min}$
- 5) Calcolo della penale per ritardo oltre i 10 minuti:
 - a. $\text{minutiAttesa} > 10 \rightarrow (\text{minutiAttesa} - 10) \times 0.5$
- 6) Applicazione dello sconto sulla durata:
 - a. 12h-24h $\rightarrow -25\%$
 - b. 24h $\rightarrow -50\%$
- 7) Restituzione del costo finale totale

Di seguito viene mostrato lo pseudocodice dell'algoritmo con l'analisi della complessità, durante lo svolgimento del processo vengono analizzate tutte le condizioni che servono a modificare il costo totale per poi arrivare ad avere il prezzo che l'utente dovrà pagare quando uscirà dal parcheggio; dato che non siamo in presenza di cicli ma solo istruzioni di assegnamento la complessità tempo è la più bassa possibile ovvero $O(1)$ come visto nell'Equazione 2.

$$O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(7) = O(1)$$

Equazione 2: Calcolo della complessità algoritmo calcolo importo permanenza

Alg CalcoloTariffa(utente, prenotazione, durataOra, minutiAttesa, OccupazioneParcheggio)

```
// Output
costoTotale

costoBase ← durataOre * 3
costoTotale ← costoBase

// Sconto età
if eta < 30
    costoTotale ← costoTotale * 0.85
else if eta > 60
    costoTotale ← costoTotale * 0.90
end if

// Sconto occupazione
if occupazione == "studente"
    costoTotale ← costoTotale * 0.90
else if occupazione == "lavoratore"
    costoTotale ← costoTotale * 0.90
end if

// Maggiorazione fascia oraria
if fasciaOraria == "pomeriggio"
    costoTotale ← costoTotale * 1.05
else if fasciaOraria == "sera"
    costoTotale ← costoTotale * 1.10
```

```

end if

// Maggiorazione tipo giorno
if tipoGiorno == "feriale"
    costoTotale ← costoTotale * 1.05
end if

// Fee in base all'occupazione
feeAttesa ← 0

if occupazioneParcheggio ≥ 50 && occupazioneParcheggio ≤ 80
    feeAttesa ← minutiAttesa * 0.05
else if occupazioneParcheggio > 80
    feeAttesa ← minutiAttesa * 0.10
end if

costoTotale ← costoTotale + feeAttesa

// Penale oltre 10 minuti
if minutiAttesa > 10
    penale ← (minutiAttesa - 10) * 0.5
    costoTotale ← costoTotale + penale
end if

// Sconto durata
if durataOre ≥ 12 && durataOre ≤ 24
    costoTotale ← costoTotale * 0.75
else if durataOre > 24
    costoTotale ← costoTotale * 0.50
end if

return costoTotale

```

end Alg

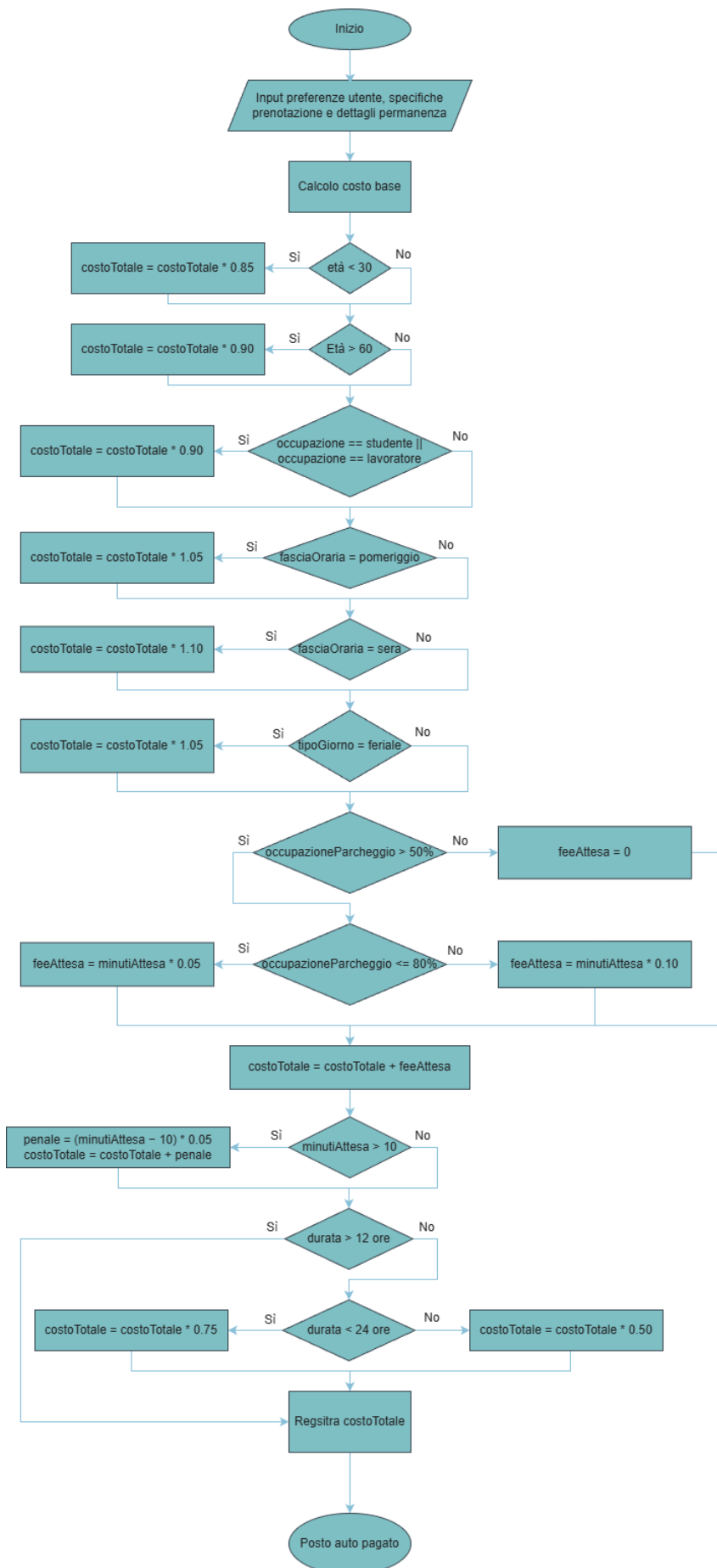


Diagramma 2: Flowchart dell'algoritmo calcolo importo permanenza

UC11: Algoritmo di identificazione percorso migliore

Breve descrizione: L'algoritmo implementa una ricerca del percorso minimo su una griglia utilizzando la tecnica di Breadth-First Search (BFS).

Dato un punto di partenza e uno di arrivo:

- Verifica la validità delle celle
- Esplora la griglia in ampiezza
- Costruisce il percorso minimo (se esiste)
- Restituisce il percorso in coordinate normalizzate

Se il percorso non esiste o i dati non sono validi, restituisce una posizione di fallback.

Attori coinvolti: Sistema

Trigger: Utente arriva al parcheggio

Postcondizione: Se esiste un percorso viene restituito il percorso con numero di passi minimo, altrimenti viene restituito un percorso di fallback

Passi dell'algoritmo:

- 1) Validazione input:
 - a. Controlla che start e goal siano dentro la griglia
 - b. Verifica che siano celle camminabili
 - c. In caso contrario → ritorno immediato
- 2) Inizializzazione BFS:
 - a. Inserisce start in:
 - i. coda (queue)
 - ii. insieme visitati (visited)
 - b. Inizializza parent[start] = null
 - c. Definisce le 4 direzioni di movimento
- 3) Esplorazione:
 - a. Finché la coda non è vuota:
 - i. Estrae il nodo corrente
 - ii. Se è il goal → termina
 - iii. Esplora i 4 vicini:
 1. Controlla validità
 2. Verifica se visitati
 3. Li aggiunge a:
 - a. visited
 - b. queue
 - c. parent
- 4) Verifica raggiungibilità:

- a. Controlla se il goal è stato raggiunto
- 5) Ricostruzione percorso:
 - a. Parte dal goal
 - b. Risale tramite parent
 - c. Costruisce la lista delle celle
- 6) Ordinamento percorso:
 - a. Inverte la lista per ottenere start $\rightarrow$ goal
- 7) Conversione finale:
 - a. Converta ogni cella in coordinate normalizzate

In figura x è mostrato lo pseudocodice dell'algoritmo con l'analisi della complessità, inizialmente vengono eseguiti controlli costanti sui dati di input, successivamente la griglia viene esplorata tramite Breadth-First Search per trovare il percorso minimo e infine, il percorso viene ricostruito e convertito in coordinate normalizzate. Poiché sono presenti cicli che possono visitare tutte le celle, la complessità dipende dalla dimensione della griglia.

$$O(1) + O(1) + O(1) + O(1) + O(N) + O(1) + O(N) + O(1) + O(1) + O(1) + O(1) + O(1) +$$
$$+ O(N) + O(1) + O(1) + O(N) = O(4N + 12) = O(N)$$

Equazione 3: Calcolo della complessità algoritmo di identificazione percorso migliore

Alg identificazionePercorsoMigliore(startC, startR, goalC, goalR)

```
// Output

percorsoNormalizzato // lista di celle normalizzate

start ← (startC, startR)

goal ← (goalC, goalR)

// Complessità  $O(1)$ 

if start fuori griglia
    return [entryPointNormalized()]
end if

// Complessità  $O(1)$ 

if goal fuori griglia
    return [centro(start)]
end if

// Complessità  $O(1)$ 

if start not camminabile
```

```

        return [centro(start)]
    end if

// Complessità  $O(1)$ 

    if goal not camminabile
        return [centro(start)]
    end if

    queue ← lista con start
    visited ← insieme con start
    parent ← mappa con (start → null)
    dirs ← [(1,0), (-1,0), (0,1), (0,-1)]
    qi ← 0

    // Esplorazione

// Complessità  $O(N)$  dove  $N$  = numero celle

    while qi < lunghezza(queue)
        current ← queue[qi]
        qi ← qi + 1

        // Complessità  $O(1)$ 

        if current == goal
            break
        end if

        // Complessità  $O(N)$ 

        for ogni direzione d in dirs
            nc ← current.c + d.c
            nr ← current.r + d.r

            // Complessità  $O(1)$ 

            if fuori griglia
                continua
            end if
        end for
    end while

```

// Complessità $O(1)$

if not camminabile

continua

end if

next $\leftarrow$ (nc, nr)

// Complessità $O(1)$

if next già visitato

continua

end if

// Complessità $O(1)$

aggiungi next a visited

parent[next] $\leftarrow$ current

aggiungi next a queue

end for

end while

// Complessità $O(1)$

if goal not in parent

return [centro(start)]

end if

// Ricostruzione percorso

cells $\leftarrow$ lista vuota

cur $\leftarrow$ goal

// Complessità $O(N)$

while cur is not null

aggiungi cur a cells

cur $\leftarrow$ parent[cur]

end while

// Ordinamento

// Complessità $O(1)$

ordered $\leftarrow$ reverse(cells)

// Complessità $O(1)$

percorsoNormalizzato $\leftarrow$ lista vuota

// Complessità $O(N)$

for cella in ordered

aggiungi centro(cella) a percorsoNormalizzato

end for

return percorsoNormalizzato

end Alg

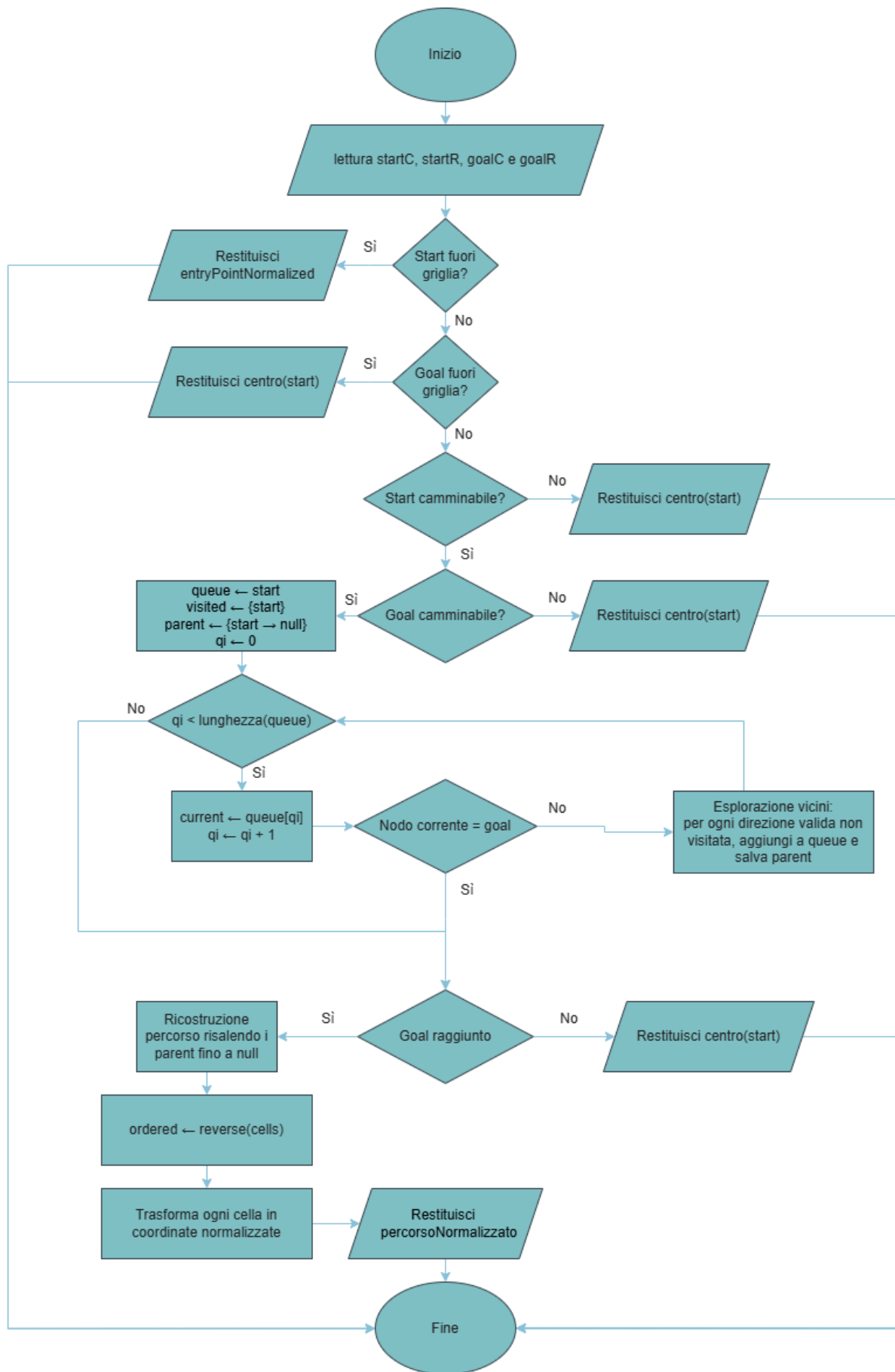


Diagramma 3: Flowchart dell'algoritmo identificazione percorso migliore

Diagramma delle componenti UML

I casi d'uso scelti per la terza iterazione sono stati aggiunti al component diagram della seconda iterazione.

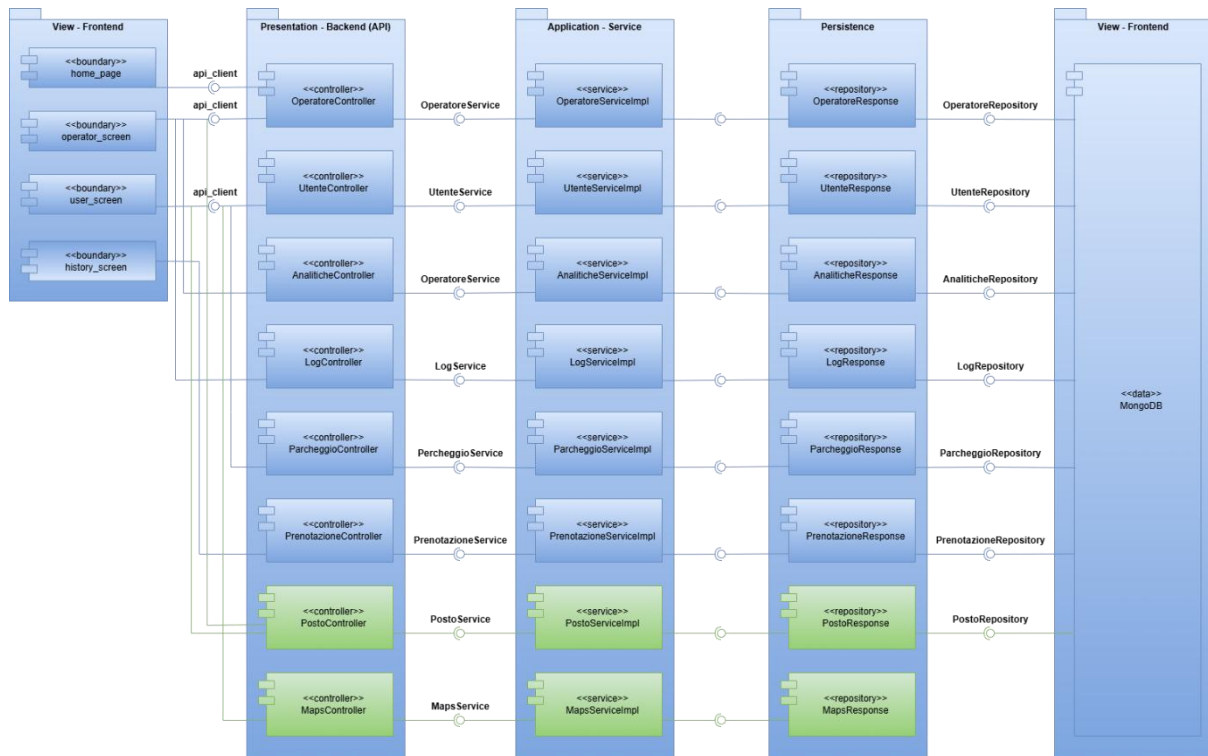


Figura 30: Diagramma delle componenti UML

Diagramma delle classi per le interfacce

Il diagramma rappresenta le interfacce introdotte durante lo sviluppo della terza iterazione.



Figura 11: Diagramma delle classi UML per interfacce

Diagramma delle classi per tipi di dato

Il tipo di dato Posto e StatoPrenotazione vengono inseriti nel Diagramma per tipo di dato dell'iterazione precedente, il nuovo diagramma dispone le nuove interazioni tra i dati.

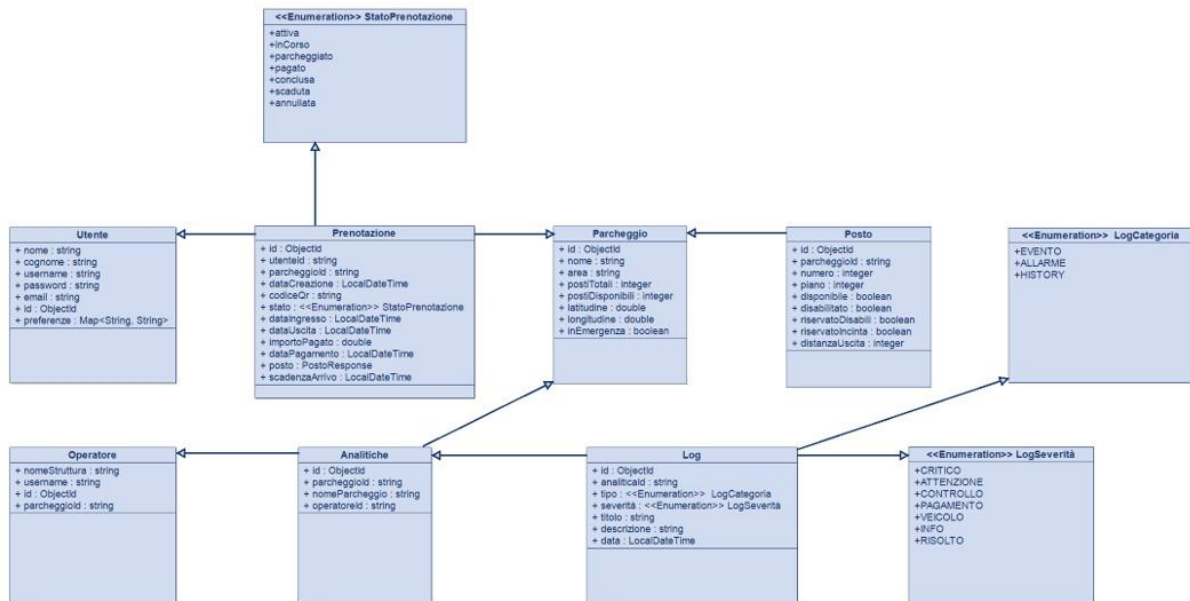


Figura 12: Diagramma delle classi UML per tipi di dato

Diagramma di deployment

I nuovi componenti definiti in questa terza iterazione sono stati inseriti nel Deployment Diagram dell'iterazione numero due.

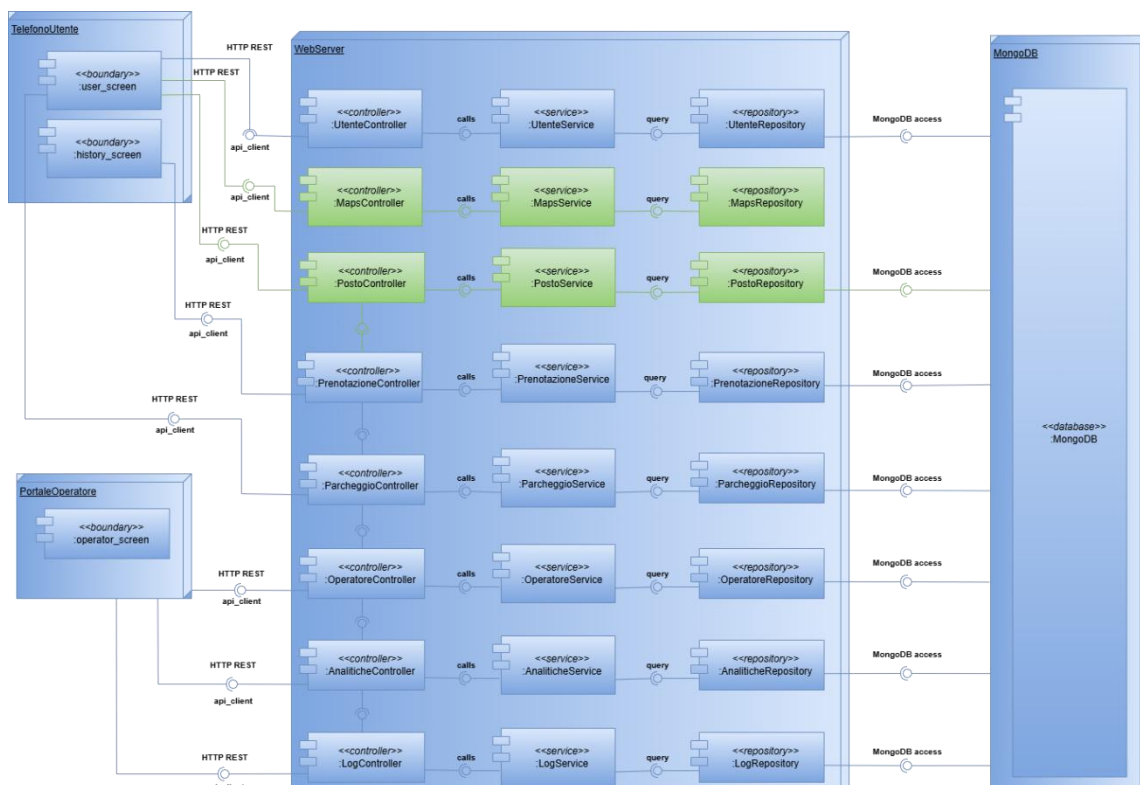


Figura 13: Diagramma di deployment UML

Testing

Analisi statica

Per garantire l'alta qualità, la manutenibilità e la sicurezza del codice sviluppato durante l'iterazione 3 del progetto, è stata eseguita un'analisi statica integrata.

Il tool scelto è **SonarLint** (plugin di SonarQube), utilizzato in Eclipse. SonarLint ha identificato e guidato la risoluzione di *Code Smells*, potenziali *Bug* e violazioni delle *best practice* di programmazione.

Per supportare una corretta gestione dei possibili errori, all'interno del package "pmg.backend.exception", sono state create una serie di classi per gestire i diversi tipi di eccezioni che si possono verificare all'interno.

Analisi dinamica

Per l'iterazione 3 sono stati effettuati aggiornamenti alle classi di test precedentemente implementate e sono state inoltre create le nuove istanze di verifica per le classi Posti ed Exception.

JUnit test

1) PostoController

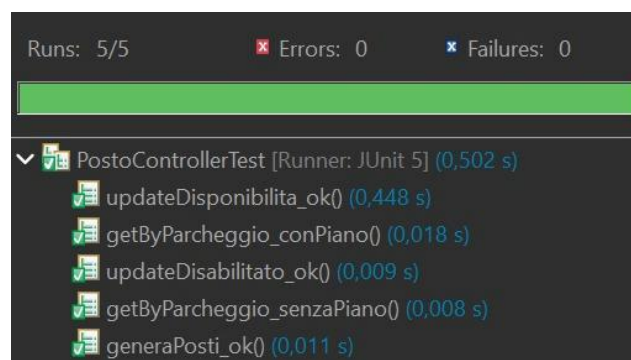


Figura 14: Esito test JUnit sulla classe PostoController

2) PostoServiceImpl

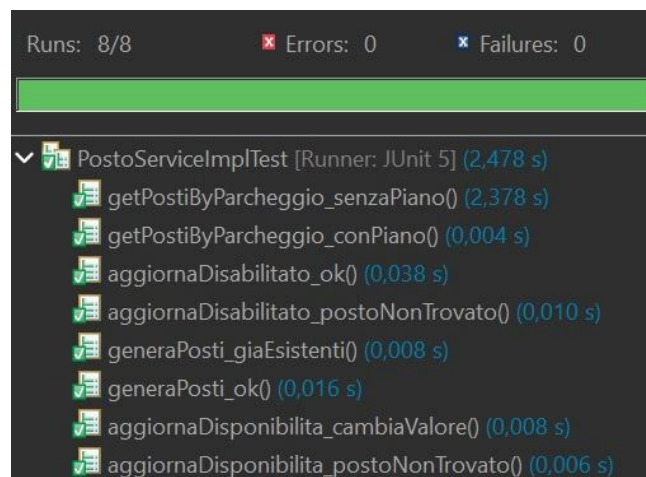


Figura 15: Esito test JUnit sulla classe PostoServiceImpl

3) PrenotazioneController

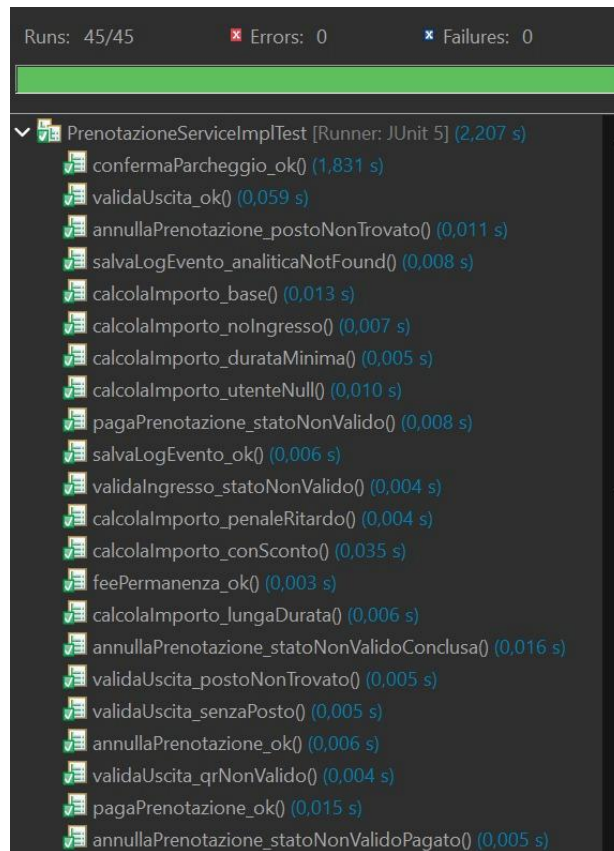


Figura 16: Esito test JUnit sulla classe PrenotazioneController

4) PrenotazioneServiceImpl

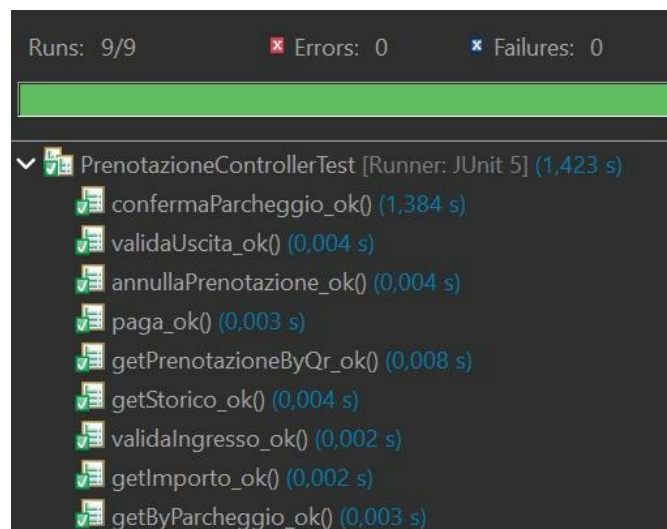


Figura 17: Esito test JUnit sulla classe PrenotazioneServiceImpl

Coverage

✓ PMG-backend		96,7 %	13.590	467	14.057
> src/main/java		95,1 %	4.702	241	4.943
> src/test/java		97,5 %	8.888	226	9.114

Figura 18: Coverage fornita dai test JUnit del progetto

Integration test dell'algoritmo

Per i due algoritmi presenti nel backend ovvero quello per la selezione del posto ottimale e quello relativo al pagamento totale che l'automobilista dovrà pagare alla fine della permanenza nel parcheggio sono state create delle classi di test apposta nelle rispettive cartelle di applicazione e sono stati creati dei casi di test appositi per controllare il corretto funzionamento dell'intero codice, oltre alla relativa integrazione con gli altri componenti, il tutto eseguito tramite JUnit.

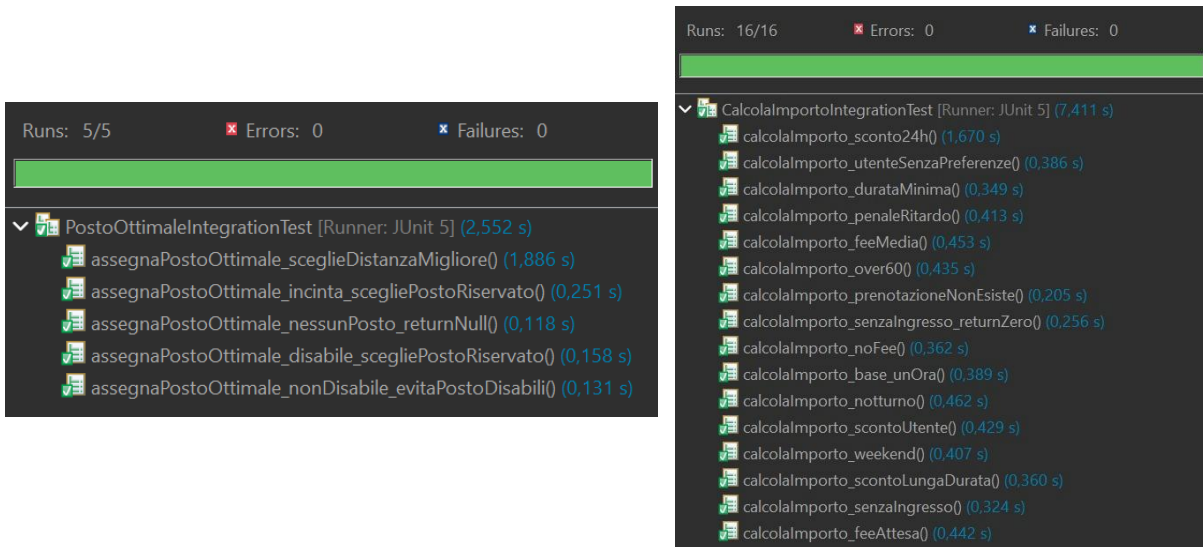


Figura 19: Esito integration test JUnit sulle classi degli algoritmi

Mentre per quanto riguarda l'algoritmo presente nella sezione frontend, ovvero il BFS che mostra all'utente la strada più breve da raggiungere partendo dall'entrata del parcheggio arrivando al posto designato, è stata creata una classe di test in flutter e testata seguendo la logica e la formattazione nativa del linguaggio.

```
PS C:\Users\ALEGI\Documents\GitHub\ParkM-G\codice\PMG-frontend> flutter test test/bfs_test.dart
00:03 +5: All tests passed!
```

Figura 20: Esito integration test Flutter sulla classe dell'algoritmo

Tutti e tre gli algoritmi hanno riscontrato un ottimo livello di successo e non presentano errori nella collaborazione con i diversi artefatti presenti nel codice sia lato backend sia lato frontend.

Documentazione API

Queste sono le API sviluppate nella prima iterazione, è possibile prenderne visione tramite la collezione Postman presente nel repository GitHub:

- API per la ricerca dei posti nel parcheggio
- API per aggiornare la disponibilità di un posto in un parcheggio
- API per la ricerca di prenotazioni tramite QRcode
- API per l'emissione dell'importo da pagare

- API per la ricerca delle prenotazioni nel parcheggio

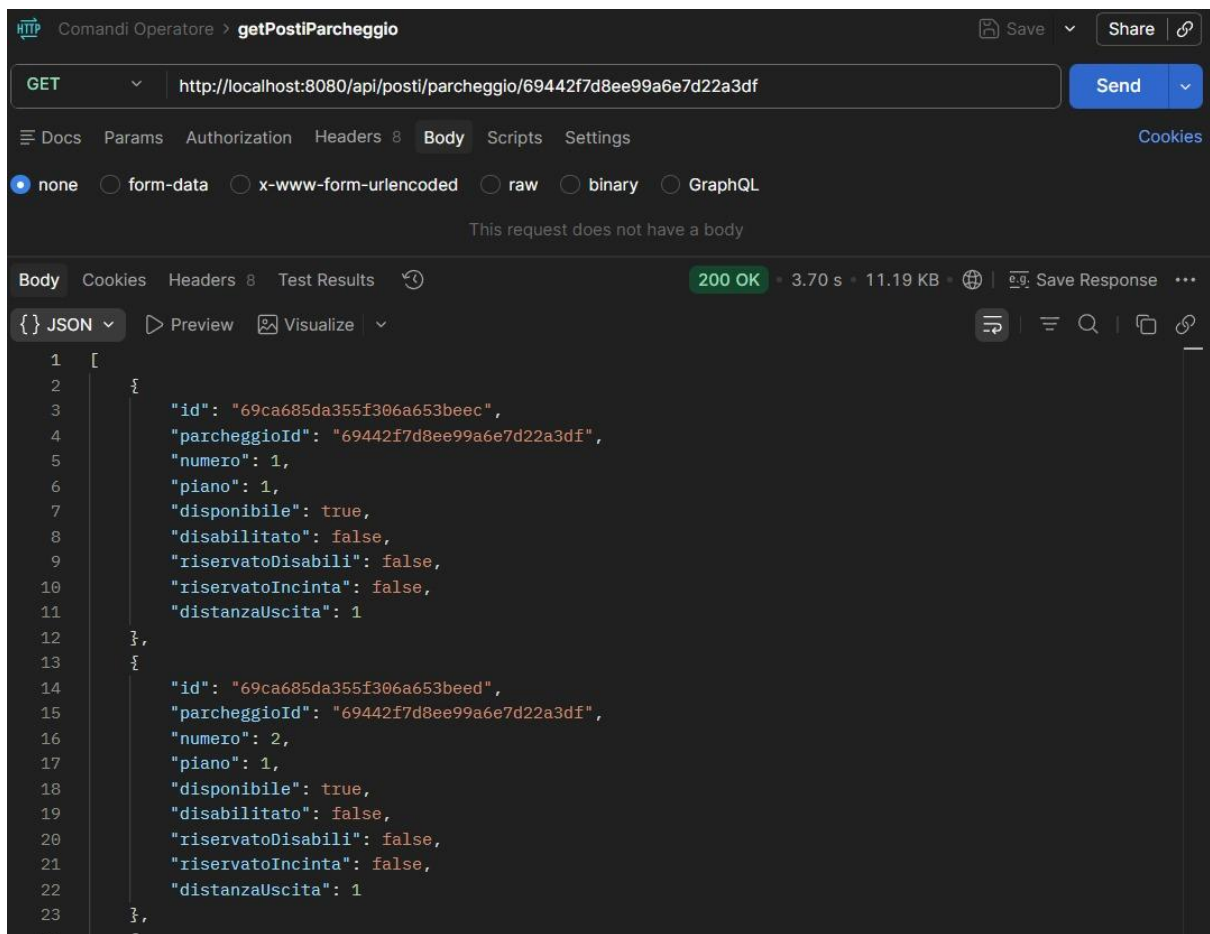


Figura 21: API per la ricerca dei posti nel parcheggio

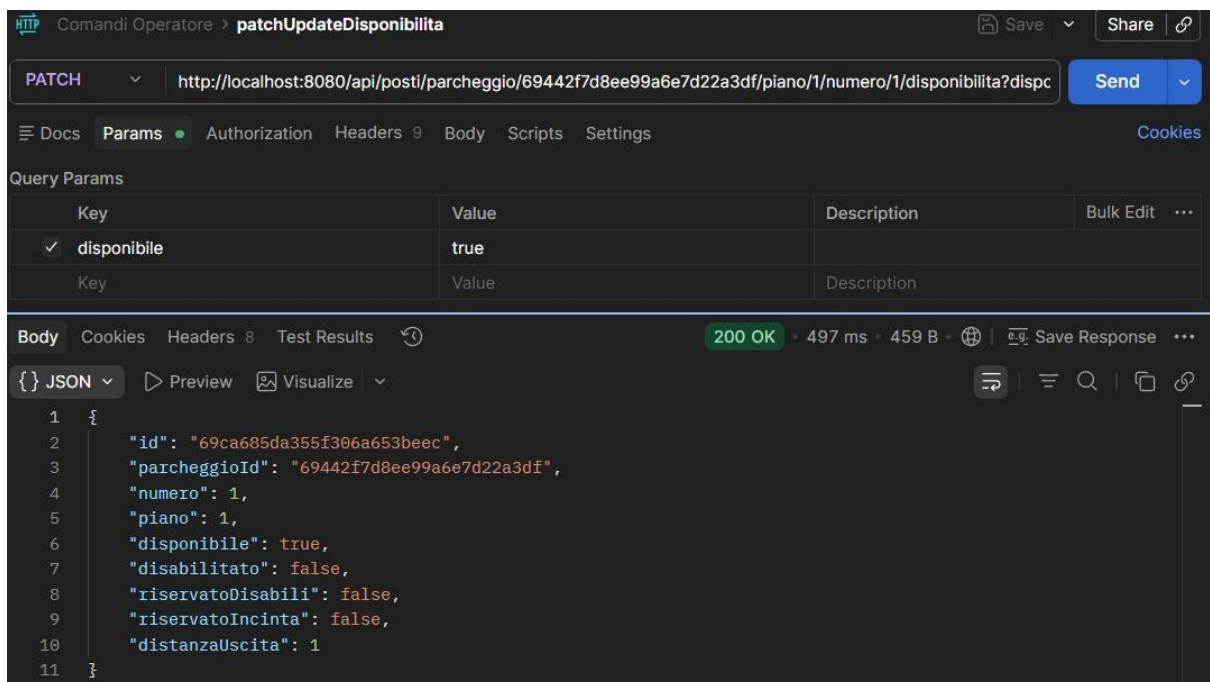


Figura 22: API per aggiornare la disponibilità di un posto in un parcheggio

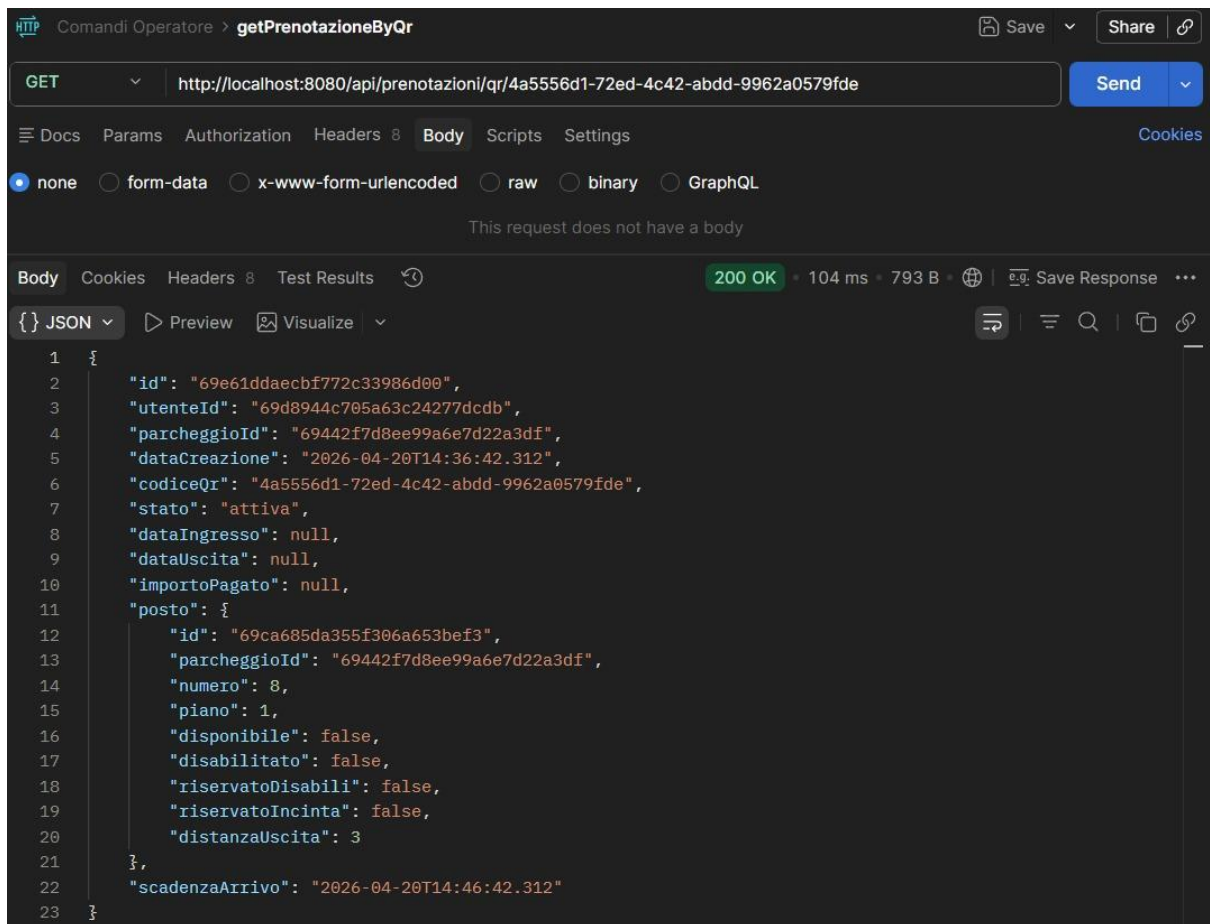


Figura 24: API per la ricerca di prenotazioni tramite QRcode

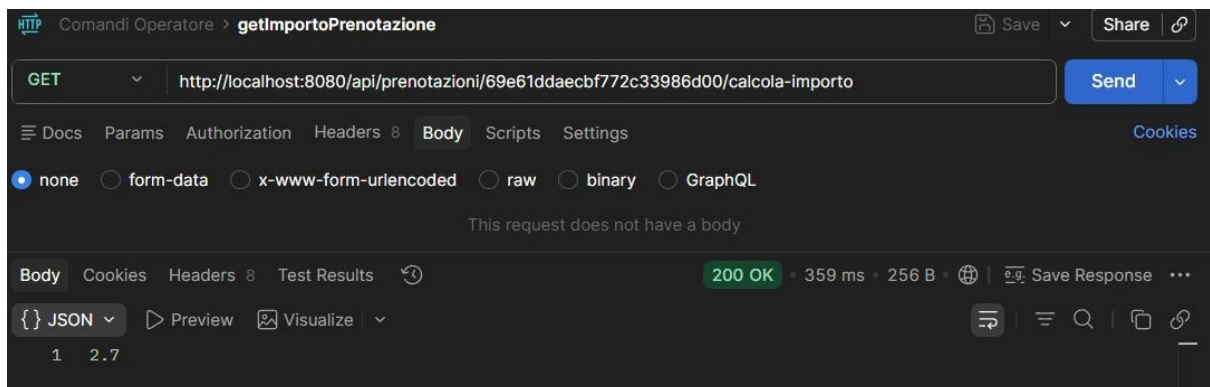


Figura 23: API per l'emissione dell'importo da pagare

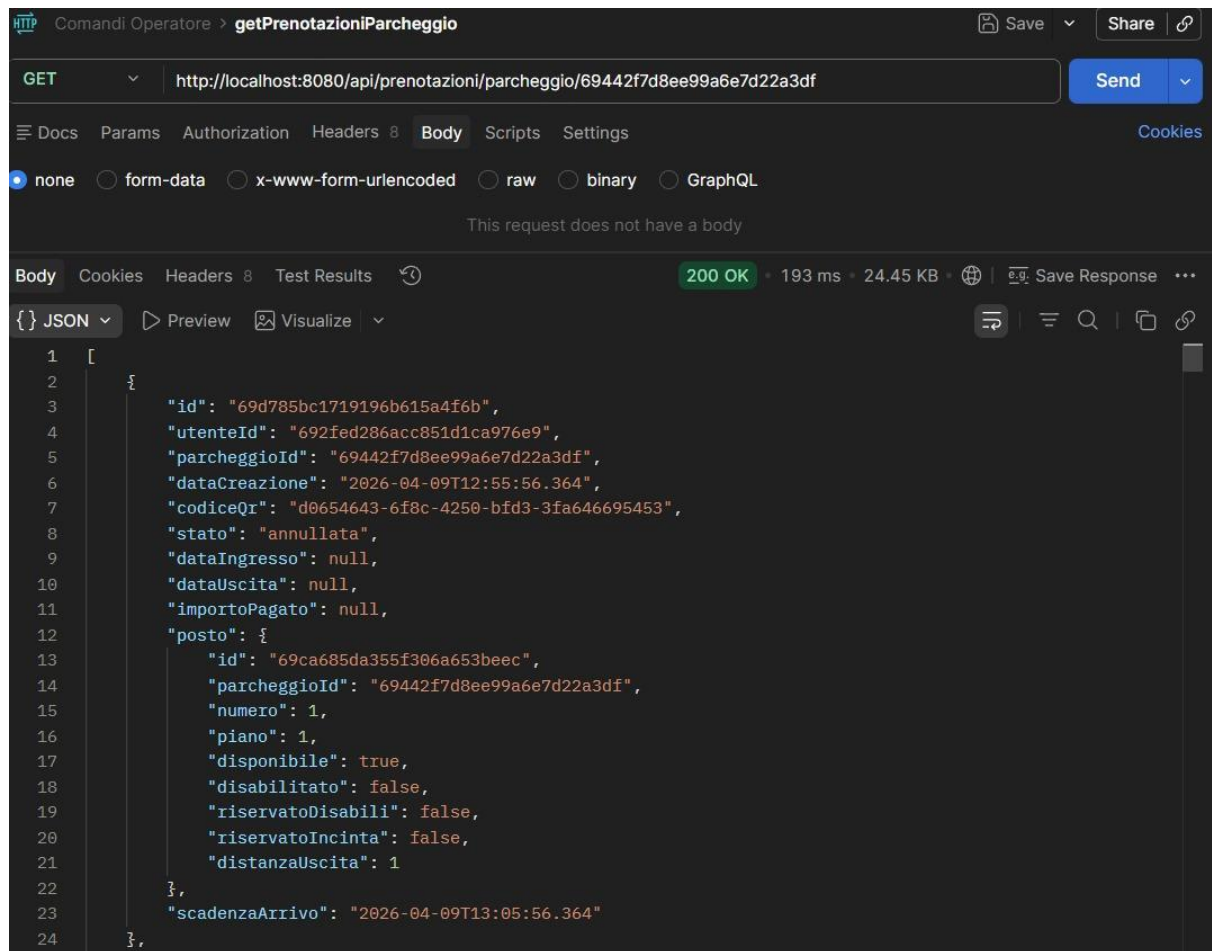


Figura 25: API per la ricerca delle prenotazioni nel parcheggio